《计算机图形学及人机交互》

实 验 报 告

学　　号　2023112621

姓　　名　程泽凯

年　　级　2023 级

专　　业　计算机科学与技术

院　　系　计算机与人工智能学院

2025~2026 第 1 学期

目录

1 实验目的、任务及要求 **3**
1.1 实验目的 . . . 3
1.2 实验任务与要求 . . . 3

2 数据结构设计 **4**
2.1 图形对象数据结构 . . . 4
2.1.1 图形基类设计 . . . 4
2.1.2 辅助几何函数 . . . 5
2.1.3 具体图形类设计 . . . 6
2.2 图形管理数据结构 . . . 20

3 文件结构设计 **20**
3.1 文件格式设计 . . . 20
3.2 各类图形的序列化格式 . . . 21
3.3 文件读取与保存实现 . . . 21

4 界面与交互设计 **25**
4.1 菜单结构设计 . . . 25
4.2 交互方式设计 . . . 25
4.2.1 绘制交互 . . . 25
4.2.2 选择与编辑交互 . . . 27
4.2.3 几何计算交互 . . . 30

5 主要功能模块结构图及功能说明 **33**
5.1 系统模块化设计思想 . . . 33
5.2 系统模块结构图 . . . 33
5.3 核心模块功能说明 . . . 33
5.3.1 主程序模块（Main Module） . . . 33
5.3.2 图形管理模块（Shape Management Module） . . . 34
5.3.3 渲染模块（Rendering Module） . . . 35
5.3.4 事件处理模块（Event Handling Module） . . . 36
5.3.5 文件管理模块（File Management Module） . . . 36
5.3.6 几何计算模块（Geometry Calculation Module） . . . 37
5.3.7 撤销/重做模块（Undo/Redo Module） . . . 38

6 用户使用说明 39

6.0.1 界面介绍 39
6.0.2 绘制基本图形 39
6.0.3 选择与编辑图形 45
6.0.4 撤销与重做操作 52
6.0.5 几何计算功能 54
6.0.6 文件操作 59

7 实验总结 63

1 实验目的、任务及要求

1.1 实验目的

通过本次实验，设计并实现一个功能完整的简单绘图软件，掌握图形交互系统的开发方法，学习图形对象的存储、管理和操作技术。具体目标包括：

1. 掌握 Direct2D 图形库的高级应用技术
2. 学习图形对象的数据结构设计与管理方法
3. 理解图形交互系统的事件处理机制
4. 掌握图形的选择、移动、旋转、缩放等变换操作
5. 学习图形计算算法（求交、垂线、圆心、切线等）
6. 掌握图形文件的保存与读取技术
7. 实现撤销/重做功能，掌握状态管理技术
8. 培养完整图形应用系统的设计与实现能力

1.2 实验任务与要求

采用 Direct2D 设计并实现一个简单的图形绘制工具软件，主要包括如下的基本功能：

任务要求

1. 基本图元的绘制：基本图元包括直线、圆、矩形、三角形、菱形、平行四边形、曲线、多义线等。基本图元的参数通过鼠标输入。
2. 选中图元：通过菜单项或工具条按钮执行” 选择” 命令。当鼠标落在某个图元上时改变鼠标形状。选中图元时以虚线形式显示，并可通过” 删除” 菜单删除该图元。
3. 移动/旋转/缩放图元：在选中某个图元之后，可通过鼠标移动/旋转/缩放该图元。旋转和缩放操作通过鼠标滚轮进行调整，支持实时预览和取消操作。
4. 完成基本的图形计算：
    - 求交：计算基本图元之间的交点，并显示交点坐标

- 绘制垂线：选择某条直线，绘制经过该点的垂线
- 求圆心：选择某个圆时显示其圆心的位置及坐标
- 求切线：点击屏幕上某点，选中某个圆，绘制该点到圆的切线，并显示切点坐标

5. 文件保存与打开：以文件的形式保存绘制的图形，并可打开已保存的文件，正常显示保存的图形。

2 数据结构设计

本系统采用面向对象的设计方法，定义了统一的图形基类和具体的图形派生类，用于存储和管理待绘制的基本图元。

2.1 图形对象数据结构

2.1.1 图形基类设计

Shape 基类是所有图形对象的抽象基类，定义了图形对象的通用接口和属性：
核心属性：

- **type**：图形类型枚举（ShapeType）
- **selected**：图形是否被选中的标志
- **color**：图形的颜色（D2D1::ColorF 类型）
- **points**：图形的关键点集合

核心方法：

- **Draw()**：纯虚函数，绘制图形
- **HitTest()**：纯虚函数，检测点是否在图形上
- **Move()**：移动图形
- **Rotate()**：旋转图形
- **Scale()**：缩放图形
- **GetCenter()**：获取图形的中心点
- **Serialize()**：序列化图形数据

- **Deserialize()**：反序列化图形数据

```
class Shape {
public:
    ShapeType type;
    bool selected;
    D2D1_COLOR_F color;
    std::vector<D2D1_POINT_2F> points;

    Shape(ShapeType t) : type(t), selected(false),
                         color(D2D1::ColorF(D2D1::ColorF::Black)) {}
    virtual ~Shape() {}

    virtual void Draw(ID2D1HwndRenderTarget* pRT,
                      ID2D1SolidColorBrush* pBrush) = 0;
    virtual bool HitTest(D2D1_POINT_2F point) = 0;
    virtual void Move(float dx, float dy) = 0;
    virtual void Rotate(D2D1_POINT_2F center, float angle) = 0;
    virtual void Scale(D2D1_POINT_2F center, float factor) = 0;
    virtual D2D1_POINT_2F GetCenter() = 0;
    virtual std::string Serialize() = 0;
    virtual void Deserialize(const std::string& data) = 0;
};
```

Listing 1: Shape 基类定义

2.1.2 辅助几何函数

为了支持图形的变换操作，定义了以下辅助函数：

```
// 点旋转函数：将点p绕center旋转angle角度（弧度制）
// 参数：p - 待旋转的点，center - 旋转中心，angle - 旋转角度（弧度）
// 返回：旋转后的新点坐标
D2D1_POINT_2F RotatePoint(D2D1_POINT_2F p, D2D1_POINT_2F center,
   float angle) {
    // 预计算sin和cos值，提高效率
    float s = sin(angle);
    float c = cos(angle);

    // 将点平移到原点（以center为新原点）
```

```
    p.x -= center.x;
    p.y -= center.y;

    // 应用旋转矩阵: [x'] = [cos -sin] [x]
    //              [y']   [sin  cos] [y]
    float xnew = p.x * c - p.y * s;  // 新x坐标
    float ynew = p.x * s + p.y * c;  // 新y坐标

    // 将点平移回原位置
    return D2D1::Point2F(xnew + center.x, ynew + center.y);
}

// 点缩放函数: 以center为中心, 按factor比例缩放点p
// 参数: p - 待缩放的点, center - 缩放中心, factor - 缩放因子(>1放大, <1缩小)
// 返回: 缩放后的新点坐标
D2D1_POINT_2F ScalePoint(D2D1_POINT_2F p, D2D1_POINT_2F center, float factor) {
    // 计算点相对于中心的偏移量, 乘以缩放因子后加回中心坐标
    // 公式: p' = center + (p - center) * factor
    return D2D1::Point2F(center.x + (p.x - center.x) * factor,
                         center.y + (p.y - center.y) * factor);
}

// 点到线段距离函数: 计算点p到线段ab的最短距离
// 参数: p - 目标点, a - 线段起点, b - 线段终点
// 返回: 点到线段的最短距离(像素单位)
// 用途: 用于判断鼠标点击是否命中图形边界
float PointToLineDistance(D2D1_POINT_2F p, D2D1_POINT_2F a, D2D1_POINT_2F b);
```

Listing 2: 辅助几何函数

2.1.3 具体图形类设计

本系统实现了 8 种具体图形类，每种图形类都继承自 Shape 基类并实现所有纯虚函数。下面详细介绍每种图形类的设计。

1. 直线类（LineShape）

数据成员：

- 继承自基类的 `points` 向量，存储两个点：起点 `points[0]` 和终点 `points[1]`

核心方法实现：

- **Draw()**：使用 `DrawLine()` 绘制从起点到终点的直线
- **HitTest()**：计算点到线段的距离，若距离小于 5 像素则认为命中
- **Move()**：对起点和终点同时进行平移
- **Rotate()**：以指定中心点旋转起点和终点
- **Scale()**：以指定中心点缩放起点和终点
- **GetCenter()**：返回起点和终点的中点

序列化格式：`LINE x1 y1 x2 y2`

```
class LineShape : public Shape {
public:
    // 构造函数：初始化直线类型，预分配2个点的空间
    LineShape() : Shape(ShapeType::LINE) {
        points.resize(2);  // points[0]为起点，points[1]为终点
    }

    // 绘制函数：绘制从起点到终点的直线
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
       pBrush) override {
        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);
        // 绘制直线，线宽2.0像素
        pRT->DrawLine(points[0], points[1], pBrush, 2.0f);
    }

    // 碰撞检测：判断点击点是否在直线附近
    bool HitTest(D2D1_POINT_2F point) override {
        // 计算点到线段的距离，小于5像素则认为命中
        return PointToLineDistance(point, points[0], points[1]) < 5.0
           f;
```

```
    }

    // 获取中心点：返回起点和终点的中点
    D2D1_POINT_2F GetCenter() override {
        return D2D1::Point2F((points[0].x + points[1].x) / 2,
                             (points[0].y + points[1].y) / 2);
    }
};
```

Listing 3: LineShape 类定义

2. 圆类（CircleShape）

数据成员：

- `points[0]`：圆心坐标
- `radius`：圆的半径（float 类型）

核心方法实现：

- **Draw()**：使用 `DrawEllipse()` 绘制圆形
- **HitTest()**：计算点到圆心的距离，判断是否在圆周附近（容差 5 像素）
- **Move()**：平移圆心坐标
- **Rotate()**：圆形旋转不变，仅旋转圆心位置
- **Scale()**：按比例缩放半径和圆心位置
- **GetCenter()**：返回圆心坐标

序列化格式：`CIRCLE cx cy radius`

```
class CircleShape : public Shape {
public:
    float radius;  // 圆的半径

    // 构造函数：初始化圆类型和半径
    CircleShape() : Shape(ShapeType::CIRCLE), radius(0) {
        points.resize(1);  // points[0]存储圆心坐标
    }

```

```
10      // 绘制函数：绘制圆形
11      void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
            pBrush) override {
12          // 选中时用红色，否则用原始颜色
13          pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
                color);
14          // 创建椭圆结构（radiusX和radiusY相等即为圆）
15          D2D1_ELLIPSE ellipse = D2D1::Ellipse(points[0], radius,
               radius);
16          // 绘制圆，线宽2.0像素
17          pRT->DrawEllipse(ellipse, pBrush, 2.0f);
18      }
19
20      // 碰撞检测：判断点击点是否在圆周附近
21      bool HitTest(D2D1_POINT_2F point) override {
22          // 计算点击点到圆心的距离
23          float dist = sqrt(pow(point.x - points[0].x, 2) +
24                            pow(point.y - points[0].y, 2));
25          // 距离与半径的差值小于5像素则认为在圆周附近
26          return fabs(dist - radius) < 5.0f;
27      }
28
29      // 缩放函数：缩放圆心位置和半径
30      void Scale(D2D1_POINT_2F center, float factor) override {
31          // 缩放圆心位置
32          points[0] = ScalePoint(points[0], center, factor);
33          // 按比例缩放半径
34          radius *= factor;
35      }
36  };
```

Listing 4: CircleShape 类定义

3. 矩形类（RectangleShape）

数据成员：

- `points[0]`：矩形左上角坐标
- `points[1]`：矩形右下角坐标

核心方法实现：

- **Draw()**：使用 DrawRectangle() 绘制矩形
- **HitTest()**：判断点是否在矩形边界附近（容差 5 像素）
- **Move()**：同时平移左上角和右下角
- **Rotate()**：旋转四个顶点后重新计算边界框
- **Scale()**：以中心点为基准缩放左上角和右下角
- **GetCenter()**：返回对角线中点

序列化格式：RECTANGLE x1 y1 x2 y2

```
class RectangleShape : public Shape {
public:
    // 构造函数：初始化矩形类型
    RectangleShape() : Shape(ShapeType::RECTANGLE) {
        points.resize(2);  // points[0]左上角，points[1]右下角
    }

    // 绘制函数：绘制矩形
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
       pBrush) override {
        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);
        // 创建矩形结构，确保left<right，top<bottom
        D2D1_RECT_F rect = D2D1::RectF(
            min(points[0].x, points[1].x), min(points[0].y, points
               [1].y),
            max(points[0].x, points[1].x), max(points[0].y, points
               [1].y)
        );
        // 绘制矩形，线宽2.0像素
        pRT->DrawRectangle(rect, pBrush, 2.0f);
    }

    // 碰撞检测：判断点击点是否在矩形边界附近
    bool HitTest(D2D1_POINT_2F point) override {
        // 计算矩形的四条边界
```

```
24          float left = min(points[0].x, points[1].x);
25          float right = max(points[0].x, points[1].x);
26          float top = min(points[0].y, points[1].y);
27          float bottom = max(points[0].y, points[1].y);
28
29          // 检查是否在左右边界附近（容差5像素）
30          return (fabs(point.x - left) < 5 || fabs(point.x - right) <
                5) &&
31                  (point.y >= top - 5 && point.y <= bottom + 5) ||
32                  // 或者在上下边界附近
33                  (fabs(point.y - top) < 5 || fabs(point.y - bottom) <
                        5) &&
34                  (point.x >= left - 5 && point.x <= right + 5);
35      }
36  };
```

Listing 5: RectangleShape 类定义

4. 三角形类（TriangleShape）

数据成员：

- `points[0]`：第一个顶点
- `points[1]`：第二个顶点
- `points[2]`：第三个顶点

核心方法实现：

- **Draw()**：依次连接三个顶点形成三角形
- **HitTest()**：判断点是否在三条边附近（容差 5 像素）
- **Move()**：对三个顶点同时平移
- **Rotate()**：以指定中心点旋转三个顶点
- **Scale()**：以指定中心点缩放三个顶点
- **GetCenter()**：返回三个顶点的重心（坐标平均值）

序列化格式：`TRIANGLE x1 y1 x2 y2 x3 y3`

```
class TriangleShape : public Shape {
public:
    // 构造函数：初始化三角形类型
    TriangleShape() : Shape(ShapeType::TRIANGLE) {
        points.resize(3);  // 存储三个顶点
    }

    // 绘制函数：依次连接三个顶点形成三角形
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
       pBrush) override {
        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);
        // 绘制三条边
        pRT->DrawLine(points[0], points[1], pBrush, 2.0f);  // 第一条
           边
        pRT->DrawLine(points[1], points[2], pBrush, 2.0f);  // 第二条
           边
        pRT->DrawLine(points[2], points[0], pBrush, 2.0f);  // 第三条
           边
    }

    // 碰撞检测：判断点击点是否在三角形的任意一条边附近
    bool HitTest(D2D1_POINT_2F point) override {
        // 检查点到三条边的距离，任意一条小于5像素则命中
        return PointToLineDistance(point, points[0], points[1]) < 5.0
           f ||
               PointToLineDistance(point, points[1], points[2]) < 5.0
                  f ||
               PointToLineDistance(point, points[2], points[0]) < 5.0
                  f;
    }

    // 获取中心点：返回三个顶点的重心（坐标平均值）
    D2D1_POINT_2F GetCenter() override {
        return D2D1::Point2F(
            (points[0].x + points[1].x + points[2].x) / 3,
            (points[0].y + points[1].y + points[2].y) / 3
        );
```

```
    }
};
```

Listing 6: TriangleShape 类定义

5. 菱形类（DiamondShape）

数据成员：

- `points[0]`：菱形中心点
- `width`：菱形宽度（水平对角线长度）
- `height`：菱形高度（垂直对角线长度）

核心方法实现：

- **Draw()**：根据中心点和宽高计算四个顶点，依次连接
- **HitTest()**：判断点是否在菱形四条边附近
- **Move()**：平移中心点
- **Rotate()**：旋转中心点位置（菱形本身形状保持）
- **Scale()**：按比例缩放宽度、高度和中心点位置
- **GetCenter()**：返回中心点

顶点计算公式：

- 上顶点：$(cx, cy - height/2)$
- 右顶点：$(cx + width/2, cy)$
- 下顶点：$(cx, cy + height/2)$
- 左顶点：$(cx - width/2, cy)$

序列化格式：`DIAMOND cx cy width height`

```
class DiamondShape : public Shape {
public:
    float width, height;  // 菱形的宽度和高度（对角线长度）

    // 构造函数：初始化菱形类型和尺寸
```

```
    DiamondShape() : Shape(ShapeType::DIAMOND), width(0), height(0) {
        points.resize(1);  // points[0]存储菱形中心点
    }

    // 绘制函数：根据中心点和宽高计算四个顶点并绘制
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
       pBrush) override {
        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);

        // 根据中心点和宽高计算四个顶点坐标
        D2D1_POINT_2F top = {points[0].x, points[0].y - height/2};
                // 上顶点
        D2D1_POINT_2F right = {points[0].x + width/2, points[0].y};
               // 右顶点
        D2D1_POINT_2F bottom = {points[0].x, points[0].y + height/2};
              // 下顶点
        D2D1_POINT_2F left = {points[0].x - width/2, points[0].y};
                // 左顶点

        // 依次连接四个顶点形成菱形
        pRT->DrawLine(top, right, pBrush, 2.0f);
        pRT->DrawLine(right, bottom, pBrush, 2.0f);
        pRT->DrawLine(bottom, left, pBrush, 2.0f);
        pRT->DrawLine(left, top, pBrush, 2.0f);
    }

    // 缩放函数：缩放中心点位置和菱形尺寸
    void Scale(D2D1_POINT_2F center, float factor) override {
        // 缩放中心点位置
        points[0] = ScalePoint(points[0], center, factor);
        // 按比例缩放宽度和高度
        width *= factor;
        height *= factor;
    }
};
```

Listing 7: DiamondShape 类定义

6. 平行四边形类（ParallelogramShape）

数据成员：

- `points[0]`：第一个顶点
- `points[1]`：第二个顶点
- `points[2]`：第三个顶点
- `points[3]`：第四个顶点（根据平行四边形性质自动计算）

核心方法实现：

- **Draw()**：依次连接四个顶点形成平行四边形
- **HitTest()**：判断点是否在四条边附近
- **Move()**：对四个顶点同时平移
- **Rotate()**：以指定中心点旋转四个顶点
- **Scale()**：以指定中心点缩放四个顶点
- **GetCenter()**：返回对角线交点（四个顶点坐标平均值）

第四个顶点计算：根据平行四边形性质，$p_4 = p_1 + p_3 - p_2$
序列化格式：`PARALLELOGRAM x1 y1 x2 y2 x3 y3 x4 y4`

```
class ParallelogramShape : public Shape {
public:
    // 构造函数：初始化平行四边形类型
    ParallelogramShape() : Shape(ShapeType::PARALLELOGRAM) {
        points.resize(4);  // 存储四个顶点
    }

    // 绘制函数：依次连接四个顶点形成平行四边形
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
        pBrush) override {
        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);
        // 绘制四条边
        pRT->DrawLine(points[0], points[1], pBrush, 2.0f);  // 第一条
```

```
            边
14          pRT->DrawLine(points[1], points[2], pBrush, 2.0f);  // 第二条
                边
15          pRT->DrawLine(points[2], points[3], pBrush, 2.0f);  // 第三条
                边
16          pRT->DrawLine(points[3], points[0], pBrush, 2.0f);  // 第四条
                边
17      }
18
19      // 碰撞检测：判断点击点是否在平行四边形的任意一条边附近
20      bool HitTest(D2D1_POINT_2F point) override {
21          // 检查点到四条边的距离，任意一条小于5像素则命中
22          return PointToLineDistance(point, points[0], points[1]) < 5.0
                f ||
23                 PointToLineDistance(point, points[1], points[2]) < 5.0
                    f ||
24                 PointToLineDistance(point, points[2], points[3]) < 5.0
                    f ||
25                 PointToLineDistance(point, points[3], points[0]) < 5.0
                    f;
26      }
27
28      // 获取中心点：返回四个顶点的几何中心（对角线交点）
29      D2D1_POINT_2F GetCenter() override {
30          return D2D1::Point2F(
31              (points[0].x + points[1].x + points[2].x + points[3].x) /
                    4,
32              (points[0].y + points[1].y + points[2].y + points[3].y) /
                    4
33          );
34      }
35  };
```

Listing 8: ParallelogramShape 类定义

7. 曲线类（CurveShape）

数据成员：

- points：存储曲线上所有控制点的向量

核心方法实现：

- **Draw()**：使用 Bezier 曲线或样条曲线连接所有控制点，形成平滑曲线
- **HitTest()**：判断点是否在曲线附近（检查与每段曲线的距离）
- **Move()**：对所有控制点同时平移
- **Rotate()**：以指定中心点旋转所有控制点
- **Scale()**：以指定中心点缩放所有控制点
- **GetCenter()**：返回所有控制点的几何中心

绘制方式：使用 Direct2D 的 `DrawGeometry()` 配合 `ID2D1PathGeometry` 绘制平滑曲线
序列化格式：`CURVE n x1 y1 x2 y2 ... xn yn`（n 为控制点数量）

```
class CurveShape : public Shape {
public:
    // 构造函数：初始化曲线类型
    CurveShape() : Shape(ShapeType::CURVE) {}

    // 绘制函数：使用Bezier曲线绘制平滑曲线
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
        pBrush) override {
        if (points.size() < 2) return;  // 至少需要2个点才能绘制

        // 选中时用红色，否则用原始颜色
        pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
            color);

        // 使用Direct2D路径几何绘制平滑Bezier曲线
        ID2D1PathGeometry* pPathGeometry = nullptr;
        ID2D1GeometrySink* pSink = nullptr;

        // 创建路径几何对象并通过GeometrySink添加曲线段
        // ...（具体实现省略）
    }

    // 碰撞检测：判断点击点是否在曲线附近
    bool HitTest(D2D1_POINT_2F point) override {
        // 遍历所有相邻控制点对，检查点到连线的距离
```

```
        for (size_t i = 0; i < points.size() - 1; i++) {
            // 任意一段距离小于5像素则命中
            if (PointToLineDistance(point, points[i], points[i+1]) <
                5.0f)
                return true;
        }
        return false;
    }
};
```

Listing 9: CurveShape 类定义

8. 多义线类（PolylineShape）

数据成员：

- points：存储多义线所有顶点的向量

核心方法实现：

- **Draw()**：依次用直线连接所有顶点
- **HitTest()**：判断点是否在任意一条线段附近
- **Move()**：对所有顶点同时平移
- **Rotate()**：以指定中心点旋转所有顶点
- **Scale()**：以指定中心点缩放所有顶点
- **GetCenter()**：返回所有顶点的几何中心

与曲线的区别：多义线用直线段连接各点，曲线用平滑曲线连接各点
序列化格式：POLYLINE n x1 y1 x2 y2 ... xn yn（n 为顶点数量）

```
class PolylineShape : public Shape {
public:
    // 构造函数：初始化多义线类型
    PolylineShape() : Shape(ShapeType::POLYLINE) {}

    // 绘制函数：用直线段依次连接所有顶点
    void Draw(ID2D1HwndRenderTarget* pRT, ID2D1SolidColorBrush*
        pBrush) override {
        if (points.size() < 2) return;  // 至少需要2个点才能绘制
```

```
9
10          // 选中时用红色，否则用原始颜色
11          pBrush->SetColor(selected ? D2D1::ColorF(D2D1::ColorF::Red) :
                color);
12
13          // 遍历所有相邻点对，用直线连接
14          for (size_t i = 0; i < points.size() - 1; i++) {
15              pRT->DrawLine(points[i], points[i+1], pBrush, 2.0f);
16          }
17      }
18
19      // 碰撞检测：判断点击点是否在多义线的任意一条线段附近
20      bool HitTest(D2D1_POINT_2F point) override {
21          // 遍历所有线段，检查点到线段的距离
22          for (size_t i = 0; i < points.size() - 1; i++) {
23              // 任意一段距离小于5像素则命中
24              if (PointToLineDistance(point, points[i], points[i+1]) <
                    5.0f)
25                  return true;
26          }
27          return false;
28      }
29
30      // 获取中心点：返回所有顶点的几何中心（坐标平均值）
31      D2D1_POINT_2F GetCenter() override {
32          float sumX = 0, sumY = 0;
33          // 累加所有点的坐标
34          for (const auto& p : points) {
35              sumX += p.x;
36              sumY += p.y;
37          }
38          // 返回平均值作为中心点
39          return D2D1::Point2F(sumX / points.size(), sumY / points.size
                ());
40      }
41  };
```

Listing 10: PolylineShape 类定义

表 1: 各图形类特点对比

图形类型	顶点数	额外参数	可变顶点	特殊性质
直线	2	无	否	最简单的图形
圆	1	半径	否	旋转不变性
矩形	2	无	否	轴对齐矩形
三角形	3	无	否	三点确定
菱形	1	宽、高	否	中心对称
平行四边形	4	无	否	对边平行
曲线	可变	无	是	平滑连接
多义线	可变	无	是	直线连接

2.2 图形管理数据结构

图形容器

使用 STL 智能指针和容器管理所有图形对象：

```
// 存储所有图形对象的智能指针
std::vector<std::shared_ptr<Shape>> shapes;

// 当前选中的图形
std::shared_ptr<Shape> selectedShape = nullptr;
```

优点：

- 自动内存管理：使用 shared_ptr 自动管理内存，避免内存泄漏
- 动态管理：支持运行时添加、删除图形
- 多态支持：通过基类指针统一管理不同类型的图形

3 文件结构设计

3.1 文件格式设计

文件格式规范

文件扩展名：.drw (Drawing File)

文件结构：

<图形数量>
<图形类型> <参数1> <参数2> ... <参数N>
<图形类型> <参数1> <参数2> ... <参数N>

...

示例文件：

```
3
LINE 100 100 200 200
CIRCLE 300 300 50
RECTANGLE 400 100 500 200
```

3.2 各类图形的序列化格式

表 2: 图形序列化格式

图形类型	序列化格式
直线	LINE x1 y1 x2 y2
圆	CIRCLE cx cy radius
矩形	RECTANGLE x1 y1 x2 y2
三角形	TRIANGLE x1 y1 x2 y2 x3 y3
菱形	DIAMOND cx cy width height
平行四边形	PARALLELOGRAM x1 y1 x2 y2 x3 y3 x4 y4
曲线	CURVE n x1 y1 x2 y2 ... xn yn
多义线	POLYLINE n x1 y1 x2 y2 ... xn yn

3.3 文件读取与保存实现

文件操作核心函数
实现文件的保存和读取功能，支持将图形数据持久化存储和恢复。
主要功能：

- **SaveToFile()**：将所有图形序列化并保存到文件
- **LoadFromFile()**：从文件读取数据并反序列化为图形对象

```
// ==================== 文件保存函数 ====================
// 功能: 将当前画布上的所有图形保存到指定文件
// 参数: filename - 保存的文件路径（.drw格式）
// 返回: true表示保存成功, false表示保存失败
bool SaveToFile(const std::string& filename) {
    // 创建输出文件流
    std::ofstream outFile(filename);
    if (!outFile.is_open()) {
        // 文件打开失败, 显示错误信息
```

```
10          MessageBox(NULL, L"无法创建文件！", L"错误", MB_OK |
                MB_ICONERROR);
11          return false;
12      }
13
14      // 第一行：写入图形总数
15      outFile << shapes.size() << std::endl;
16
17      // 遍历所有图形对象，逐个序列化
18      for (const auto& shape : shapes) {
19          // 调用每个图形的Serialize()方法，获取文本表示
20          std::string data = shape->Serialize();
21          // 将序列化数据写入文件（每个图形占一行）
22          outFile << data << std::endl;
23      }
24
25      // 关闭文件流
26      outFile.close();
27
28      // 显示保存成功提示
29      MessageBox(NULL, L"文件保存成功！", L"提示", MB_OK |
            MB_ICONINFORMATION);
30      return true;
31  }
32
33  // ==================== 文件读取函数 ====================
34  // 功能：从指定文件加载图形数据并重建图形对象
35  // 参数：filename - 要打开的文件路径（.drw格式）
36  // 返回：true表示读取成功，false表示读取失败
37  bool LoadFromFile(const std::string& filename) {
38      // 创建输入文件流
39      std::ifstream inFile(filename);
40      if (!inFile.is_open()) {
41          // 文件打开失败，显示错误信息
42          MessageBox(NULL, L"无法打开文件！", L"错误", MB_OK |
                MB_ICONERROR);
43          return false;
44      }
45
46      // 清空当前画布上的所有图形
```

```
47      shapes.clear();
48      selectedShape = nullptr;
49
50      // 第一行：读取图形总数
51      int count;
52      inFile >> count;
53      inFile.ignore();  // 忽略换行符
54
55      // 循环读取每个图形
56      for (int i = 0; i < count; i++) {
57          // 读取一行数据
58          std::string line;
59          std::getline(inFile, line);
60
61          // 使用字符串流解析数据
62          std::istringstream iss(line);
63          std::string shapeType;
64          iss >> shapeType;  // 读取图形类型（第一个单词）
65
66          // 根据图形类型创建对应的图形对象
67          std::shared_ptr<Shape> shape;
68          if (shapeType == "LINE") {
69              shape = std::make_shared<LineShape>();
70          } else if (shapeType == "CIRCLE") {
71              shape = std::make_shared<CircleShape>();
72          } else if (shapeType == "RECTANGLE") {
73              shape = std::make_shared<RectangleShape>();
74          } else if (shapeType == "TRIANGLE") {
75              shape = std::make_shared<TriangleShape>();
76          } else if (shapeType == "DIAMOND") {
77              shape = std::make_shared<DiamondShape>();
78          } else if (shapeType == "PARALLELOGRAM") {
79              shape = std::make_shared<ParallelogramShape>();
80          } else if (shapeType == "CURVE") {
81              shape = std::make_shared<CurveShape>();
82          } else if (shapeType == "POLYLINE") {
83              shape = std::make_shared<PolylineShape>();
84          } else {
85              // 未知图形类型，跳过该行
86              continue;
```

```
87          }
88
89          // 调用图形的反序列化方法，解析参数数据
90          shape->Deserialize(line);
91
92          // 将图形添加到容器中
93          shapes.push_back(shape);
94      }
95
96      // 关闭文件流
97      inFile.close();
98
99      // 刷新画布显示
100     InvalidateRect(hwnd, NULL, TRUE);
101
102     // 显示加载成功提示
103     MessageBox(NULL, L"文件加载成功！", L"提示", MB_OK |
            MB_ICONINFORMATION);
104     return true;
105 }
```

Listing 11: 文件读取与保存函数实现

4 界面与交互设计

4.1 菜单结构设计

表 3: 菜单结构

菜单	菜单项	功能说明
文件	新建	清空画布
	打开	从文件加载图形
	保存	保存图形到文件
	退出	退出程序
绘制	直线	绘制直线
	圆	绘制圆
	矩形	绘制矩形
	三角形	绘制三角形
	菱形	绘制菱形
	平行四边形	绘制平行四边形
	曲线	绘制曲线
	多义线	绘制多义线
编辑	选择	选择图形
	取消选择	取消当前选择
	删除	删除选中图形
	移动	移动选中图形
	旋转	旋转选中图形（滚轮调整）
	缩放	缩放选中图形（滚轮调整）
	撤销	撤销上一步操作
	重做	重做已撤销的操作
计算	求交	计算两图形交点
	垂线	绘制垂线
	圆心	显示圆心
	切线	绘制切线

4.2 交互方式设计

4.2.1 绘制交互

基本图形绘制流程：

1. 直线：点击起点 → 移动鼠标（实时预览）→ 点击终点
   - 鼠标事件：WM_LBUTTONDOWN（记录起点）→ WM_MOUSEMOVE（更新预览）→ WM_LBUTTONDOWN（确定终点）
   - 预览效果：使用虚线显示临时直线
   - 取消操作：按 ESC 键或右键点击

2. 圆：点击圆心 → 移动鼠标（实时预览）→ 点击确定半径

   - 鼠标事件：WM_LBUTTONDOWN（记录圆心）→ WM_MOUSEMOVE（计算半径并预览）→ WM_LBUTTONDOWN（确定半径）
   - 半径计算：$r = \sqrt{(x - cx)^2 + (y - cy)^2}$
   - 预览效果：使用虚线显示临时圆

3. 矩形：点击左上角 → 移动鼠标（实时预览）→ 点击右下角

   - 鼠标事件：WM_LBUTTONDOWN（记录第一个角点）→ WM_MOUSEMOVE（更新预览）→ WM_LBUTTONDOWN（确定对角点）
   - 预览效果：使用虚线显示临时矩形
   - 自动调整：系统自动处理左上、右下角的位置关系

4. 三角形：依次点击三个顶点

   - 鼠标事件：三次 WM_LBUTTONDOWN 分别记录三个顶点
   - 预览效果：前两个点用虚线连接，第三个点确定后完成三角形
   - 点击顺序：按顺时针或逆时针顺序点击

5. 菱形：点击中心 → 移动鼠标（实时预览）→ 点击确定大小

   - 鼠标事件：WM_LBUTTONDOWN（记录中心点）→ WM_MOUSEMOVE（计算宽高并预览）→ WM_LBUTTONDOWN（确定尺寸）
   - 尺寸计算：宽度 $= 2|x - cx|$，高度 $= 2|y - cy|$
   - 预览效果：使用虚线显示临时菱形

6. 平行四边形：依次点击三个顶点，第四个顶点自动计算

   - 鼠标事件：三次 WM_LBUTTONDOWN 分别记录三个顶点
   - 第四点计算：$P_4 = P_1 + P_3 - P_2$（向量加法）
   - 预览效果：前两个点用虚线连接，第三个点确定后自动补全第四个点

7. 曲线/多义线：连续点击添加点 → 右键或双击完成

   - 鼠标事件：多次 WM_LBUTTONDOWN 添加控制点 → WM_RBUTTONDOWN 或双击完成

- 最少点数：至少需要 2 个点才能形成有效的曲线/多义线
- 预览效果：每添加一个点，实时显示已连接的线段
- 曲线平滑：曲线使用 Bezier 插值，多义线使用直线连接

绘制模式通用规则：

- 模式切换：通过菜单或工具栏选择绘制模式
- 取消操作：按 ESC 键或点击右键取消当前绘制
- 实时预览：所有图形在绘制过程中都提供实时预览
- 光标提示：光标形状根据当前操作状态变化（十字、箭头、手型等）
- 状态栏提示：在状态栏显示当前操作提示信息

4.2.2 选择与编辑交互

选择交互：

- 进入选择模式：点击” 编辑 → 选择” 或按快捷键 Ctrl+S
- 碰撞检测：
  - 鼠标移动时调用 HitTest() 方法检测是否悬停在图形上
  - 检测容差：5 像素范围内认为命中
  - 悬停反馈：光标变为手型（IDC_HAND）
- 选中反馈：
  - 点击图形后设置 selected 标志为 true
  - 使用红色虚线（DashStyle）重新绘制选中的图形
  - 在状态栏显示图形类型和中心坐标
- 取消选择：点击空白区域或按 ESC 键

移动交互：

- 操作流程：
  1. 选中图形后点击” 编辑 → 移动” 或按快捷键 Ctrl+M
  2. WM_LBUTTONDOWN 记录起始鼠标位置 (x_0, y_0)

3. WM_MOUSEMOVE 计算偏移量：$dx = x - x_0$，$dy = y - y_0$
4. 调用 shape->Move(dx, dy) 更新图形所有顶点坐标
5. WM_LBUTTONUP 完成移动操作

- 实时预览：拖动过程中实时更新图形位置
- 边界检测：可选择是否限制图形在画布范围内移动
- 撤销支持：移动操作可撤销

旋转交互：

- 操作流程：
  1. 选中图形后点击” 编辑 → 旋转” 或按快捷键 Ctrl+R
  2. 获取图形中心点作为旋转中心：center = shape->GetCenter()
  3. WM_MOUSEWHEEL 事件：每滚动一格旋转 5 度（$\pi/36$ 弧度）
  4. 调用 shape->Rotate(center, angle) 应用旋转变换
  5. 左键点击确认，右键点击取消并恢复原状
- 旋转方向：向上滚动逆时针旋转，向下滚动顺时针旋转
- 角度显示：在状态栏实时显示当前旋转角度
- 实时预览：旋转过程中实时更新图形显示
- 旋转矩阵：使用 2D 旋转矩阵变换所有顶点坐标

缩放交互：

- 操作流程：
  1. 选中图形后点击” 编辑 → 缩放” 或按快捷键 Ctrl+Z
  2. 获取图形中心点作为缩放中心：center = shape->GetCenter()
  3. WM_MOUSEWHEEL 事件：每滚动一格缩放 10%（factor = 1.1 或 0.9）
  4. 调用 shape->Scale(center, factor) 应用缩放变换
  5. 左键点击确认，右键点击取消并恢复原状
- 缩放方向：向上滚动放大，向下滚动缩小
- 缩放限制：设置最小缩放比例 0.1 和最大缩放比例 10.0

- 比例显示：在状态栏实时显示当前缩放比例
- 实时预览：缩放过程中实时更新图形显示

删除交互：

- 操作方式：选中图形后点击” 编辑 → 删除” 或按 Delete 键
- 确认对话框：弹出确认对话框防止误删
- 撤销支持：删除操作可撤销，图形数据保存在撤销栈中

撤销/重做交互：

- 撤销操作：
  - 点击” 编辑 → 撤销” 或按快捷键 Ctrl+Z
  - 从撤销栈中恢复上一个状态
  - 当前状态压入重做栈
- 重做操作：
  - 点击” 编辑 → 重做” 或按快捷键 Ctrl+Y
  - 从重做栈中恢复下一个状态
  - 当前状态压入撤销栈
- 历史记录：支持最多 50 步的撤销历史记录
- 状态保存：每次操作后保存整个 shapes 容器的深拷贝
- 菜单状态：根据栈状态动态启用/禁用撤销和重做菜单项

键盘快捷键汇总：

快捷键	功能
Ctrl+S	选择模式
Ctrl+M	移动模式
Ctrl+R	旋转模式
Ctrl+Z	撤销
Ctrl+Y	重做
Delete	删除选中图形
ESC	取消当前操作

4.2.3 几何计算交互

求交点：

- 操作流程：
  1. 点击” 计算 → 求交” 进入求交模式
  2. 依次点击选择两个图形（第一个图形用蓝色高亮，第二个用绿色高亮）
  3. 系统自动计算两图形的交点
  4. 在交点位置绘制红色圆点标记
  5. 在状态栏或消息框显示交点坐标
- 支持的图形组合：
  - 直线与直线：解二元一次方程组
  - 直线与圆：解二次方程，可能有 0、1 或 2 个交点
  - 圆与圆：计算圆心距，可能有 0、1 或 2 个交点
- 计算公式：
  - 直线-直线交点：求解 $\begin{cases} y = k_1 x + b_1 \\ y = k_2 x + b_2 \end{cases}$
  - 直线-圆交点：将直线方程代入圆方程 $(x - cx)^2 + (y - cy)^2 = r^2$
- 特殊情况处理：
  - 无交点：提示” 两图形无交点”
  - 无穷多交点（重合）：提示” 两图形重合”

绘制垂线：

- 操作流程：
  1. 点击” 计算 → 垂线” 进入垂线模式
  2. 点击选择一条直线（用蓝色高亮显示）
  3. 在画布上点击一个点作为垂足所在位置
  4. 系统自动计算垂线方程并绘制（绿色虚线）
  5. 标记垂足位置（绿色圆点）
- 计算方法：

  - 原直线斜率为 k，垂线斜率为 $-1/k$
  - 垂足坐标：点到直线的投影点
  - 垂足公式：$P_{foot} = P_0 + \frac{(P-P_0)\cdot\vec{d}}{|\vec{d}|^2}\vec{d}$
- 显示信息：在状态栏显示垂足坐标和垂线方程
- 特殊情况：竖直线的垂线为水平线，需特殊处理

显示圆心：

- 操作流程：
  1. 点击” 计算 → 圆心” 进入圆心模式
  2. 点击选择一个圆（用蓝色高亮显示）
  3. 系统自动在圆心位置绘制红色十字标记
  4. 显示圆心坐标和半径信息
- 显示样式：
  - 十字标记：两条垂直相交的线段，长度为 10 像素
  - 颜色：红色（RGB: 255, 0, 0）
  - 线宽：2 像素
- 信息显示：
  - 圆心坐标：(cx, cy)
  - 半径：r
  - 周长：$C = 2\pi r$
  - 面积：$S = \pi r^2$

绘制切线：

- 操作流程：
  1. 点击” 计算 → 切线” 进入切线模式
  2. 在画布上点击一个外部点 P
  3. 点击选择一个圆（用蓝色高亮显示）
  4. 系统自动计算两条切线并绘制（紫色虚线）

5. 标记两个切点位置（紫色圆点）
6. 显示切点坐标

- 计算方法：
  - 点到圆心距离：$d = \sqrt{(x - cx)^2 + (y - cy)^2}$
  - 切线长度：$l = \sqrt{d^2 - r^2}$
  - 切点角度：$\theta = \arcsin(r/d)$
  - 两个切点分别在圆心-P 连线的两侧，角度差为 2θ
- 特殊情况处理：
  - 点在圆内：提示” 点在圆内，无法绘制切线”
  - 点在圆上：只有一条切线（垂直于半径）
  - 点在圆外：有两条切线
- 显示信息：
  - 切点 1 坐标：(x_1, y_1)
  - 切点 2 坐标：(x_2, y_2)
  - 切线长度：l

几何计算通用规则：

- 结果保存：计算结果（辅助线、标记点）可选择是否保存到画布
- 颜色区分：不同类型的辅助图形使用不同颜色便于区分
- 图层管理：辅助图形显示在独立图层，可一键清除
- 精度设置：坐标显示精度为小数点后 2 位
- 取消操作：按 ESC 键退出当前计算模式

5 主要功能模块结构图及功能说明

5.1 系统模块化设计思想

本系统采用模块化设计思想，将复杂的绘图软件系统分解为多个功能独立、职责明确的模块。每个模块负责特定的功能，模块之间通过清晰的接口进行通信，实现了高内聚、低耦合的设计目标。
模块化设计的优势：

- 可维护性：各模块独立开发和测试，便于定位和修复问题
- 可扩展性：新增功能只需添加或修改相关模块，不影响其他模块
- 可复用性：模块可以在其他项目中复用
- 团队协作：不同开发人员可以并行开发不同模块

5.2 系统模块结构图

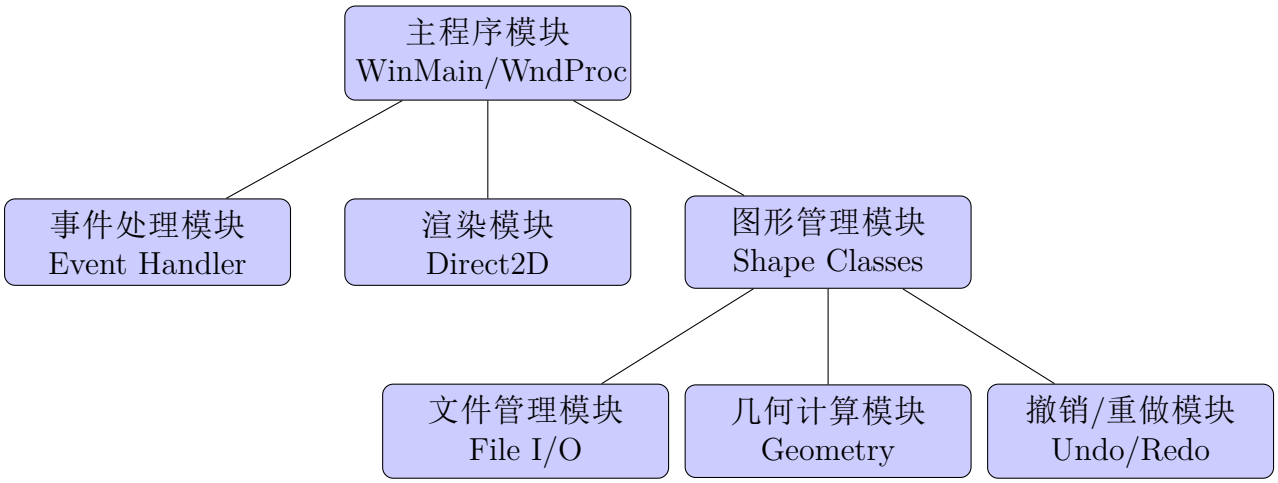


图 1: 系统模块结构图

5.3 核心模块功能说明

5.3.1 主程序模块（Main Module）

模块职责：

- Windows 窗口的创建和初始化
- Direct2D 资源的初始化和管理

- 消息循环和消息分发
- 菜单命令的处理和分发
- 协调各子模块的工作

关键函数：

- `WinMain()`：程序入口点，创建窗口并启动消息循环
- `WndProc()`：窗口过程函数，处理 Windows 消息
- `CreateDeviceResources()`：初始化 Direct2D 资源
- `DiscardDeviceResources()`：清理 Direct2D 资源

与其他模块的接口：

- 调用渲染模块的绘制函数
- 调用事件处理模块处理用户输入
- 调用文件管理模块进行文件操作

5.3.2 图形管理模块（Shape Management Module）

模块职责：

- 定义统一的图形基类接口（Shape）
- 实现 8 种具体图形类（Line, Circle, Rectangle 等）
- 管理图形对象的生命周期
- 维护图形容器（vector<shared_ptr<Shape»）
- 处理图形的选择状态
- 实现图形的变换操作（移动、旋转、缩放）

核心数据结构：

- `Shape`：图形基类（抽象类）
- `Line, Circle, RectangleShape, ...`：具体图形类

- `vector<shared_ptr<Shape>> g_shapes`：图形容器
- `Shape* g_selectedShape`：当前选中图形

对外接口：

- `Draw()`：绘制图形
- `HitTest()`：检测点击
- `Move/Rotate/Scale()`：图形变换
- `Serialize/Deserialize()`：序列化

5.3.3 渲染模块（Rendering Module）

模块职责：

- 管理 Direct2D 渲染目标和画刷
- 调度所有图形对象的绘制
- 实现虚线绘制（选中状态）
- 实现实时预览绘制
- 绘制辅助元素（交点、圆心、切线等）

关键函数：

- `OnPaint()`：主绘制函数，绘制所有图形
- `CreateDeviceResources()`：创建渲染资源
- 各图形类的 `Draw()` 方法：具体绘制实现

依赖关系：

- 依赖主程序模块提供的 Direct2D 资源
- 从图形管理模块获取待绘制的图形列表

5.3.4 事件处理模块（Event Handling Module）

模块职责：

- 处理鼠标事件（点击、移动、拖动、释放、滚轮）
- 处理键盘事件
- 管理操作状态机（绘制模式、选择模式、编辑模式）
- 实现交互逻辑（绘制、选择、移动、旋转、缩放）

关键函数：

- `OnLButtonDown()`：鼠标左键按下处理
- `OnMouseMove()`：鼠标移动处理
- `OnLButtonUp()`：鼠标左键释放处理
- `OnRButtonDown()`：鼠标右键按下处理
- `WM_MOUSEWHEEL`：鼠标滚轮处理

状态管理：

- `g_currentTool`：当前操作模式（绘制/选择/编辑）
- `g_isDrawing/g_isMoving/g_isRotating/g_isScaling`：操作状态标志
- `g_calcMode`：几何计算模式

5.3.5 文件管理模块（File Management Module）

模块职责：

- 设计图形文件格式（.drw）
- 实现图形数据的序列化
- 实现图形数据的反序列化
- 处理文件对话框
- 错误处理和数据验证

关键函数：

- SaveToFile()：保存图形到文件
- LoadFromFile()：从文件加载图形
- 各图形类的 Serialize()：序列化单个图形
- 各图形类的 Deserialize()：反序列化单个图形

文件格式：

```
<图形数量>
<图形类型> <参数1> <参数2> ...
...
```

5.3.6 几何计算模块（Geometry Calculation Module）

模块职责：

- 实现直线-直线求交算法
- 实现直线-圆求交算法
- 实现圆-圆求交算法
- 实现垂线计算和绘制
- 实现圆心显示
- 实现切线计算（点到圆）
- 提供辅助几何函数（点旋转、点缩放、距离计算等）

关键函数：

- CalculateIntersections()：计算图形交点
- LineLineIntersection()：直线-直线求交
- LineCircleIntersection()：直线-圆求交
- CircleCircleIntersection()：圆-圆求交
- DrawPerpendicular()：绘制垂线
- ShowCenter()：显示圆心

- DrawTangent()：绘制切线
- RotatePoint()：旋转点
- ScalePoint()：缩放点

5.3.7 撤销/重做模块（Undo/Redo Module）

模块职责：

- 管理操作历史记录
- 实现撤销功能（最多 50 步）
- 实现重做功能
- 通过深拷贝保证状态独立性

核心数据结构：

- g_undoStack：撤销栈，存储历史状态
- g_redoStack：重做栈，存储已撤销的状态
- MAX_UNDO_STEPS = 50：最大撤销步数

关键函数：

- SaveStateToUndoStack()：保存当前状态到撤销栈
- IDM_UNDO：撤销操作处理
- IDM_REDO：重做操作处理

实现机制：

- 使用序列化/反序列化实现图形深拷贝
- 每次操作前保存当前状态
- 撤销时从栈中恢复历史状态
- 新操作会清空重做栈

6 用户使用说明

本节说明如何使用本绘图软件的各项功能。

6.1 界面介绍

界面组成：

- 菜单栏：包含” 文件”、” 绘制”、” 编辑”、” 计算” 四个主菜单
- 画布区域：白色背景的绘图区域，用于显示和操作图形
- 状态栏：（可选）显示当前操作提示和坐标信息

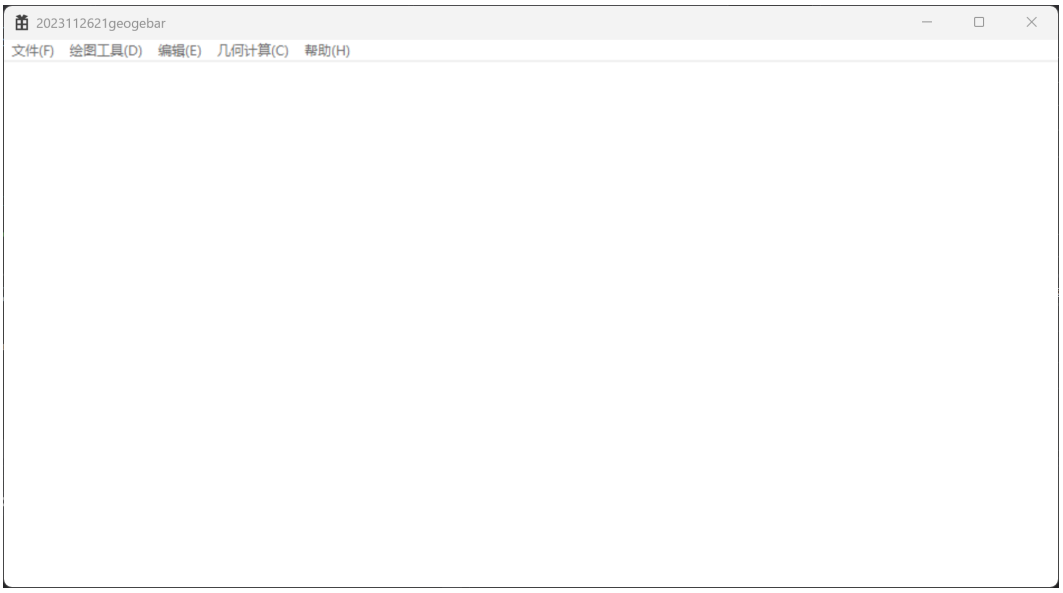


图 2: 程序界面示例

6.2 绘制基本图形

1. 绘制直线

1. 点击菜单” 绘制 → 直线”
2. 在画布上点击鼠标左键确定起点
3. 移动鼠标，可看到虚线预览
4. 再次点击鼠标左键确定终点，完成直线绘制

5. 按 ESC 键或右键可取消当前操作



图 3: 绘制直线示例

2. 绘制圆

1. 点击菜单” 绘制 → 圆”
2. 在画布上点击鼠标左键确定圆心
3. 移动鼠标，可看到圆的虚线预览，半径随鼠标位置变化
4. 再次点击鼠标左键确定半径，完成圆的绘制

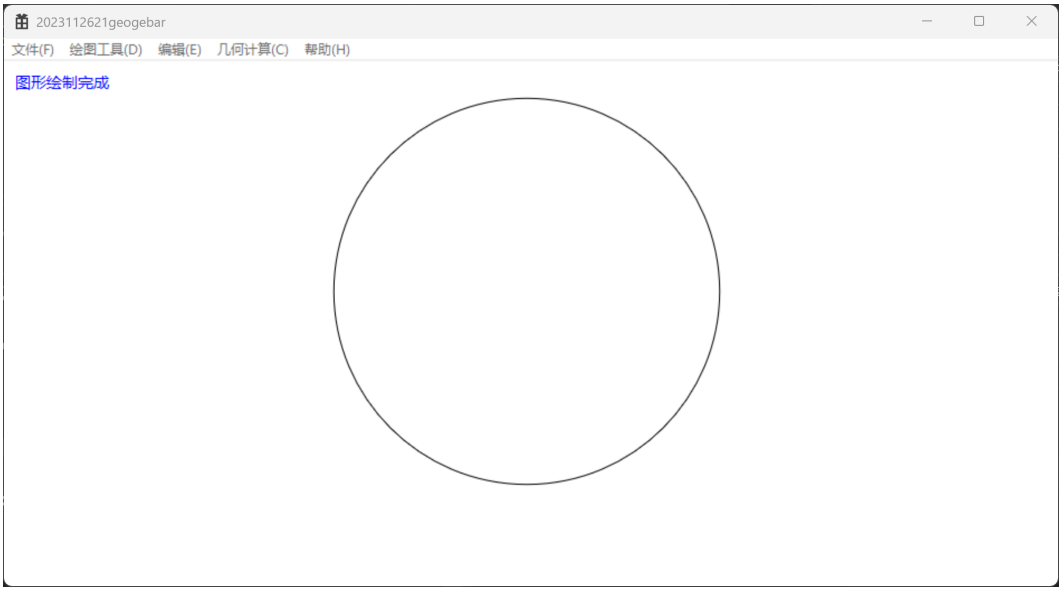


图 4: 绘制圆示例

3. 绘制矩形

1. 点击菜单” 绘制 → 矩形”
2. 在画布上点击鼠标左键确定第一个角点
3. 移动鼠标，可看到矩形的虚线预览
4. 再次点击鼠标左键确定对角点，完成矩形绘制

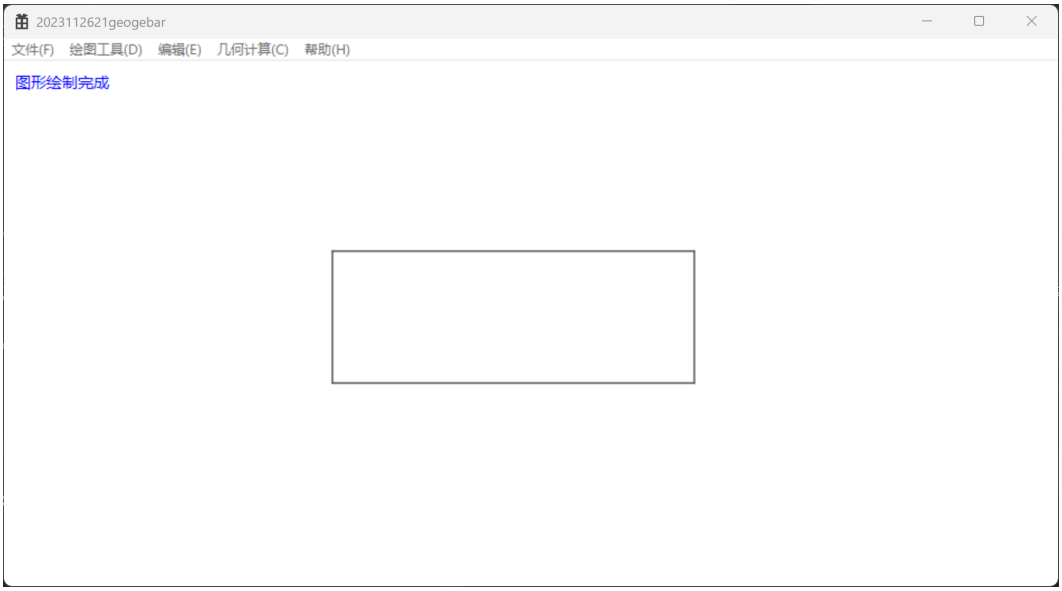


图 5: 绘制矩形示例

4. 绘制三角形

1. 点击菜单” 绘制 → 三角形”
2. 依次点击三个顶点位置
3. 第一、二个点之间会显示虚线预览
4. 点击第三个点后自动完成三角形绘制

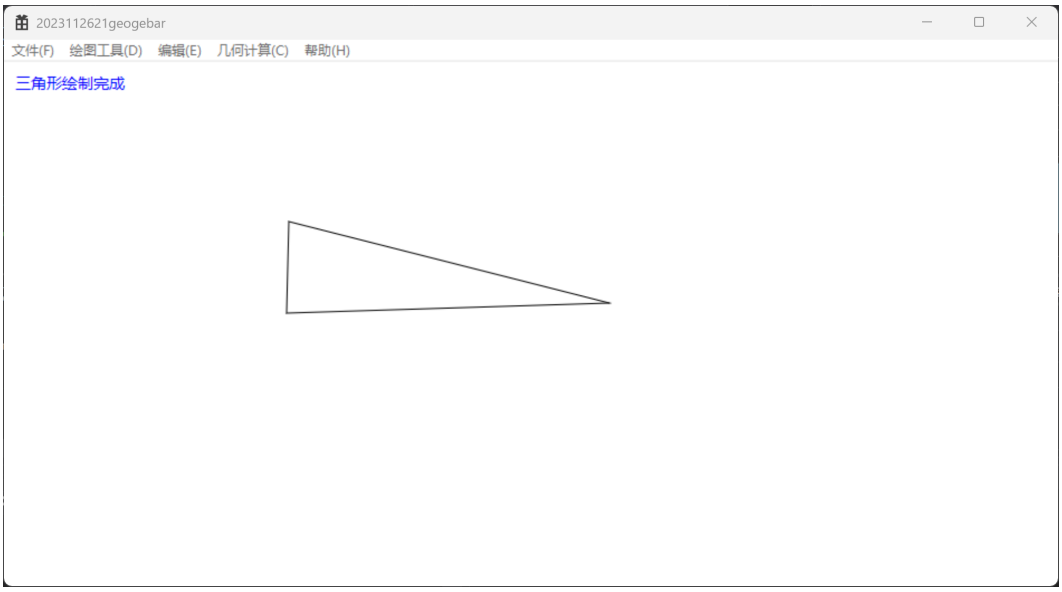


图 6: 绘制三角形示例

5. 绘制菱形

1. 点击菜单” 绘制 → 菱形”
2. 在画布上点击鼠标左键确定中心点
3. 移动鼠标，菱形大小随鼠标位置变化
4. 再次点击鼠标左键确定尺寸，完成菱形绘制

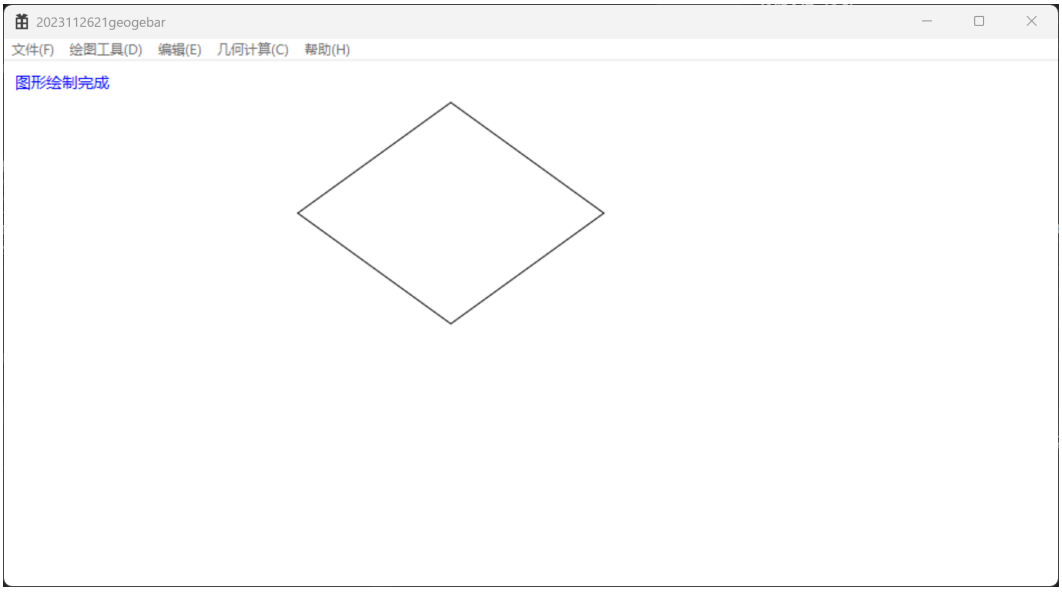


图 7: 绘制菱形示例

6. 绘制平行四边形

1. 点击菜单” 绘制 → 平行四边形”
2. 依次点击三个顶点
3. 系统自动计算并绘制第四个顶点
4. 完成平行四边形绘制

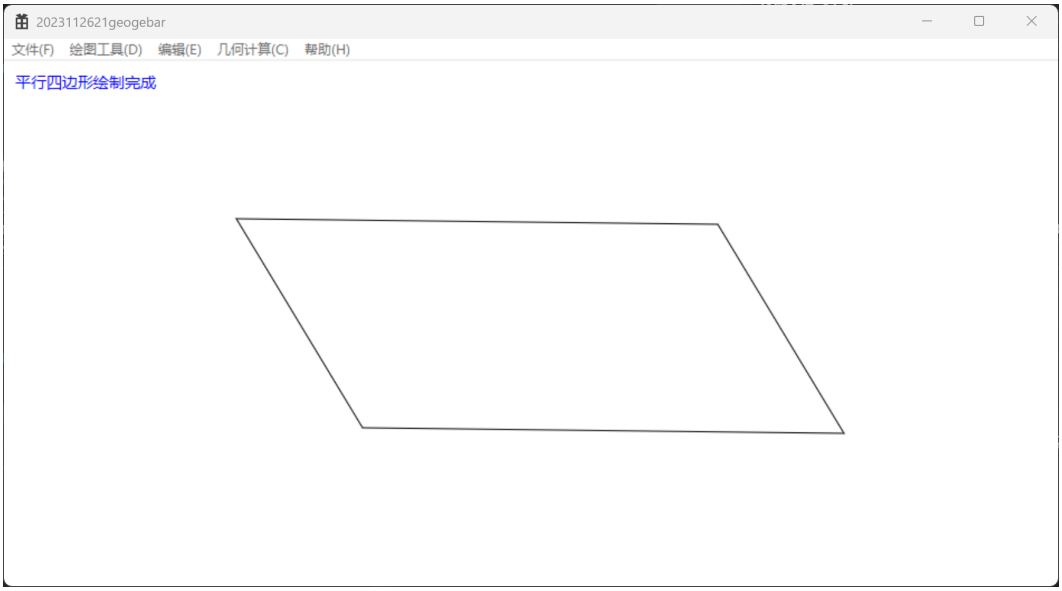


图 8: 绘制平行四边形示例

7. 绘制曲线

1. 点击菜单” 绘制 → 曲线”
2. 连续点击鼠标左键添加控制点
3. 每添加一个点，曲线会实时更新
4. 右键点击或双击完成曲线绘制
5. 至少需要 2 个点才能形成有效曲线

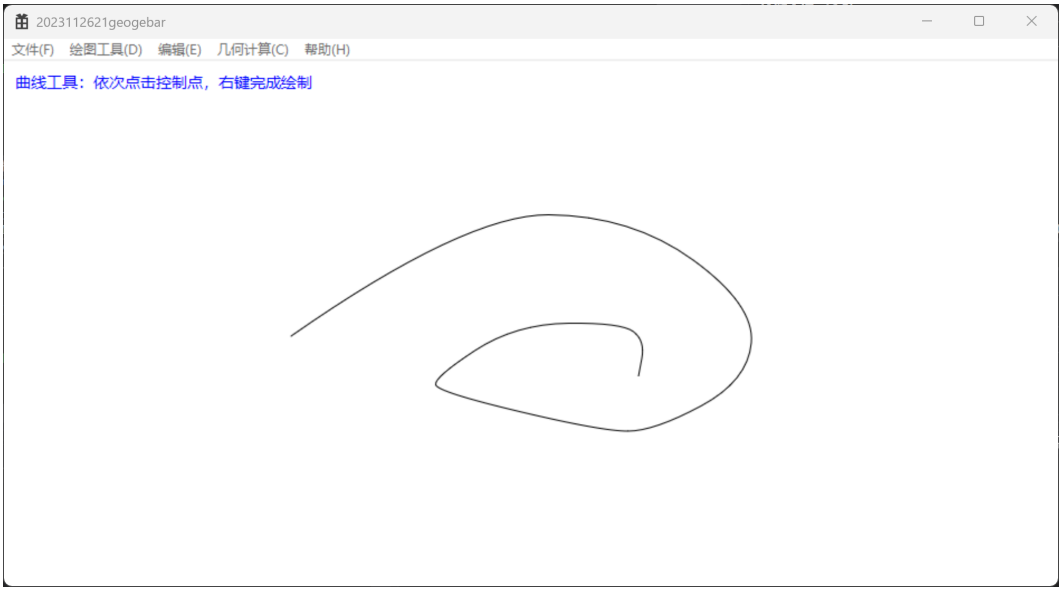


图 9: 绘制曲线示例

8. 绘制多义线

1. 点击菜单” 绘制 → 多义线”
2. 连续点击鼠标左键添加顶点
3. 每添加一个点，会用直线连接到前一个点
4. 右键点击或双击完成多义线绘制

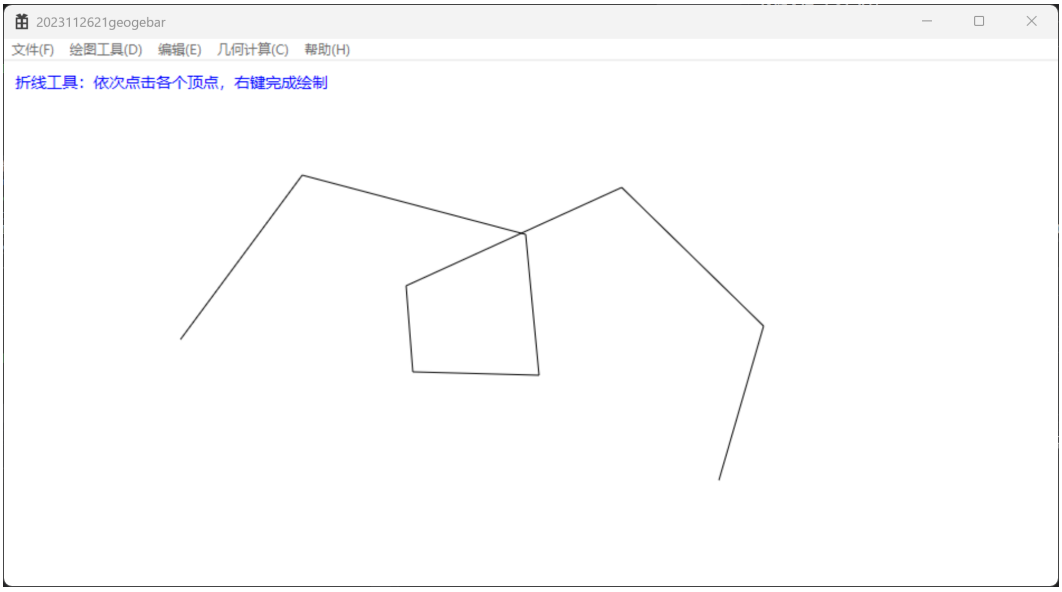


图 10: 绘制多义线示例

6.3 选择与编辑图形

1. 选择图形

1. 点击菜单” 编辑 → 选择” 进入选择模式
2. 将鼠标移动到图形上，光标变为手型
3. 点击图形，选中的图形会以红色虚线显示
4. 点击空白区域或按 ESC 键取消选择

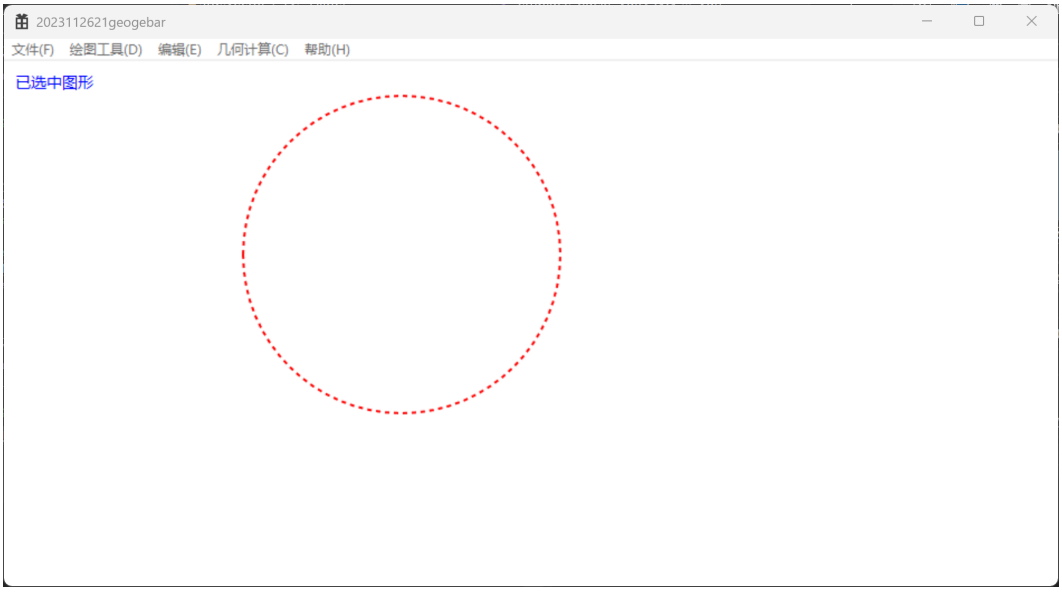


图 11: 选择图形示例

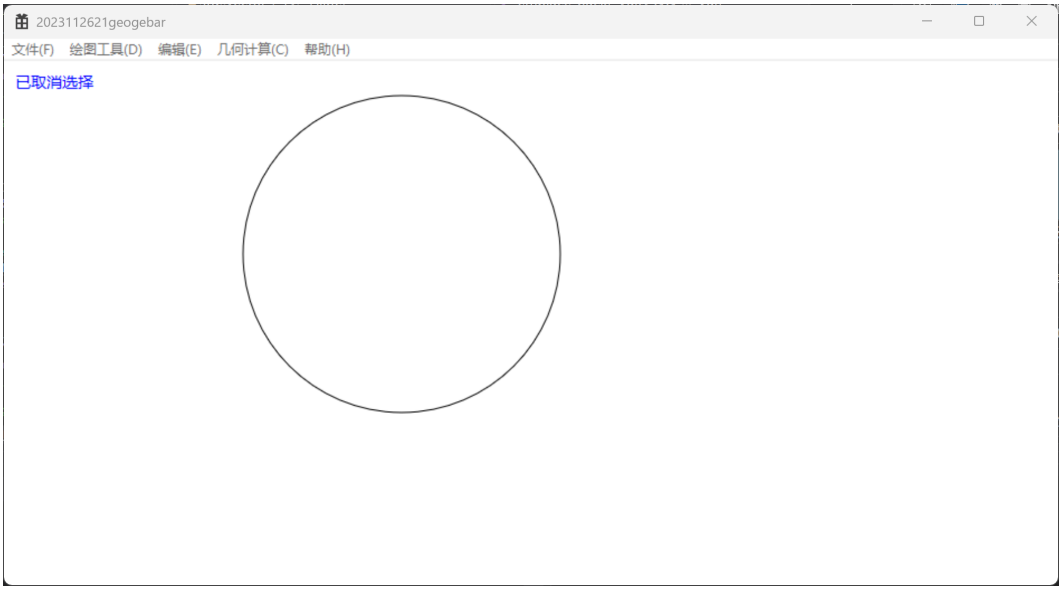


图 12: 取消选择示例

2. 移动图形

1. 先选中要移动的图形
2. 点击菜单” 编辑 → 移动”
3. 按住鼠标左键拖动图形到目标位置
4. 释放鼠标左键完成移动

5. 移动过程中图形会实时跟随鼠标移动

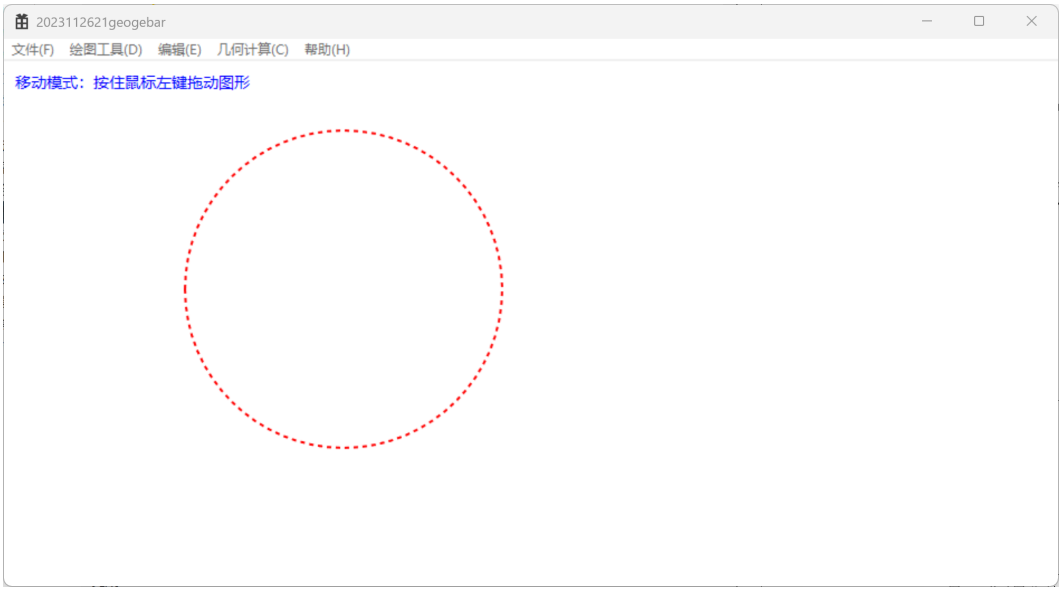


图 13: 移动图形示例

3. 旋转图形

1. 先选中要旋转的图形
2. 点击菜单” 编辑 → 旋转”
3. 滚动鼠标滚轮调整旋转角度
   - 向上滚动：逆时针旋转
   - 向下滚动：顺时针旋转
4. 旋转过程中可实时预览效果
5. 点击鼠标左键确认旋转
6. 点击鼠标右键取消旋转，恢复原状

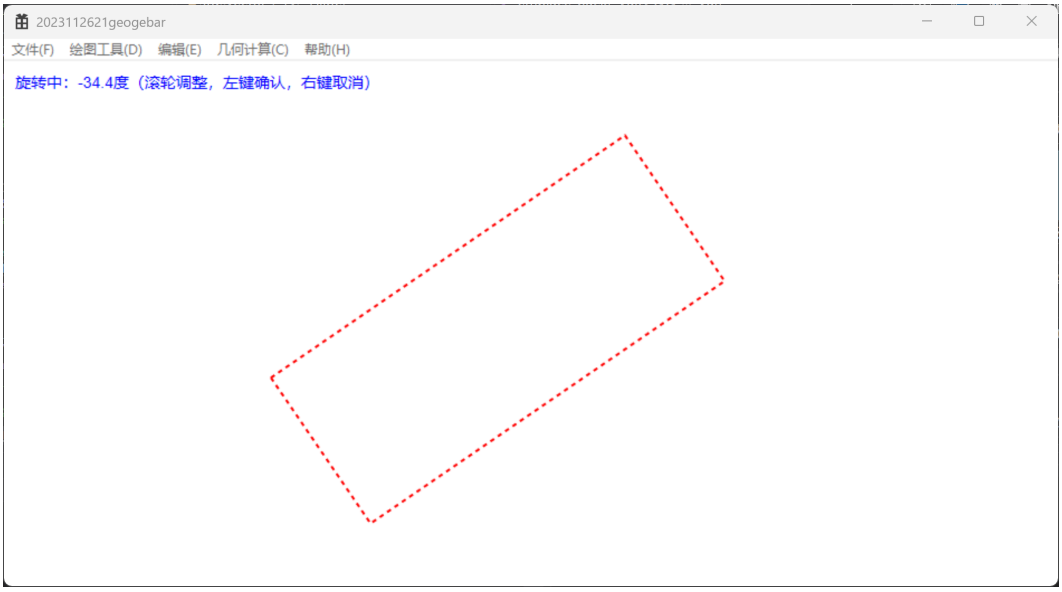


图 14: 旋转图形示例

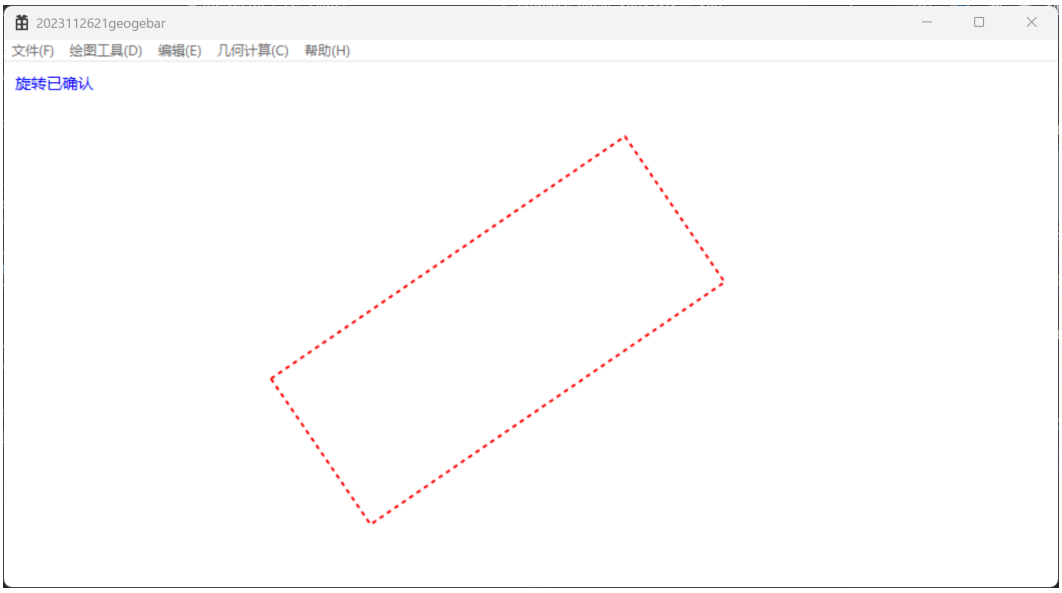


图 15: 旋转图形示例

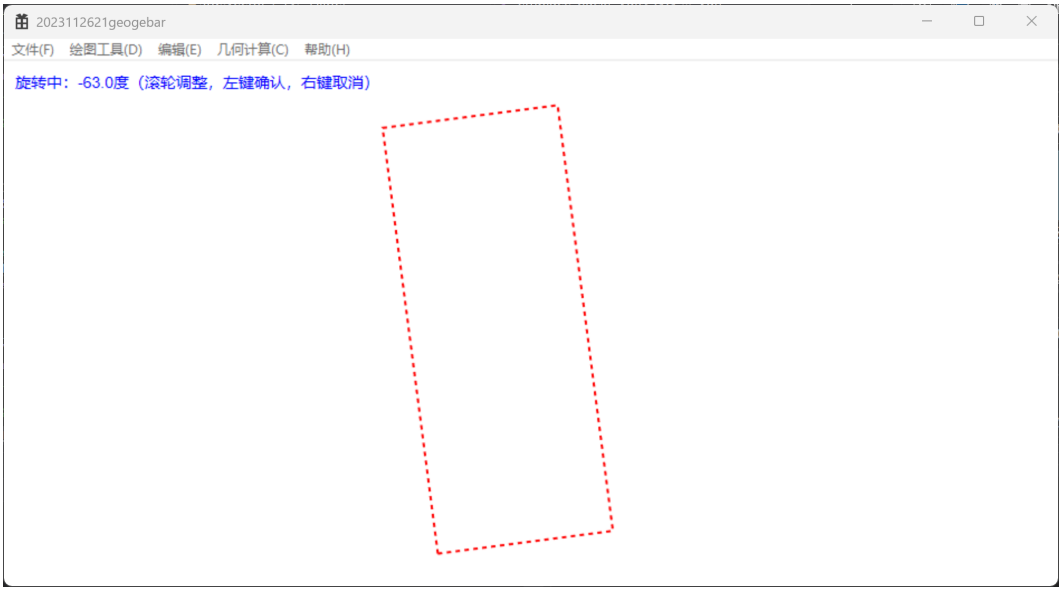


图 16: 右键取消旋转示例

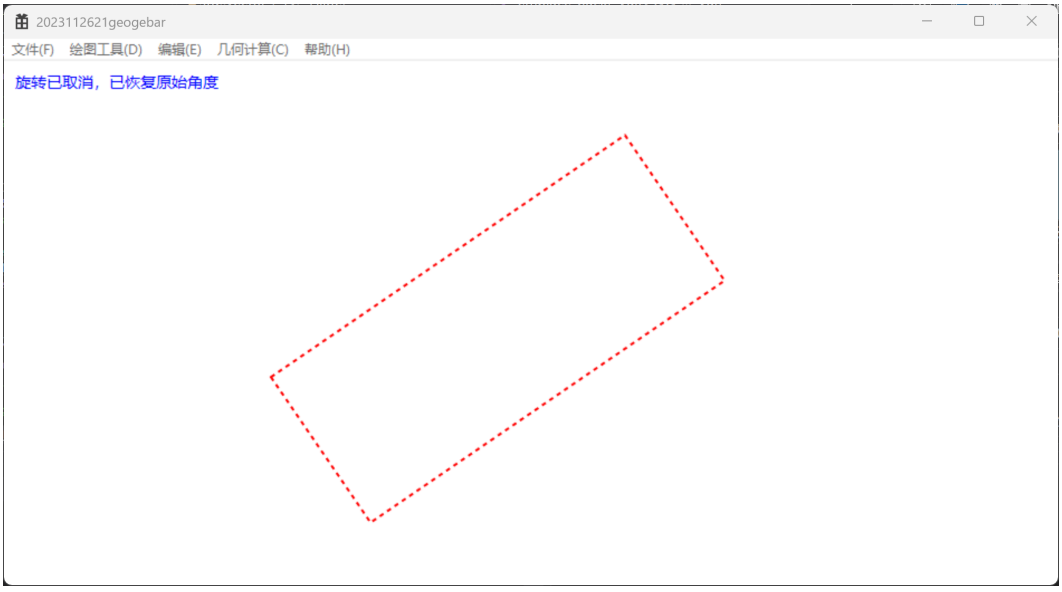


图 17: 右键取消旋转示例

4. 缩放图形

1. 先选中要缩放的图形
2. 点击菜单” 编辑 → 缩放”
3. 滚动鼠标滚轮调整缩放比例
    - 向上滚动：放大（每次增加 10%）

- 向下滚动：缩小（每次减少 10%）
- 缩放范围：0.1 倍到 10 倍

4. 缩放过程中可实时预览效果
5. 点击鼠标左键确认缩放
6. 点击鼠标右键取消缩放，恢复原状

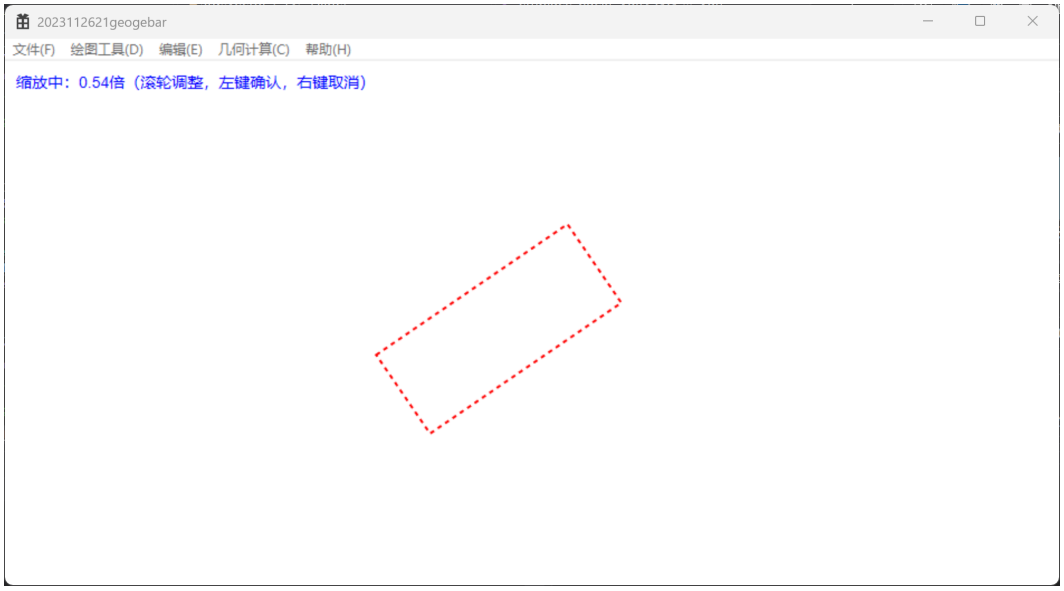


图 18: 缩放图形示例（缩小）

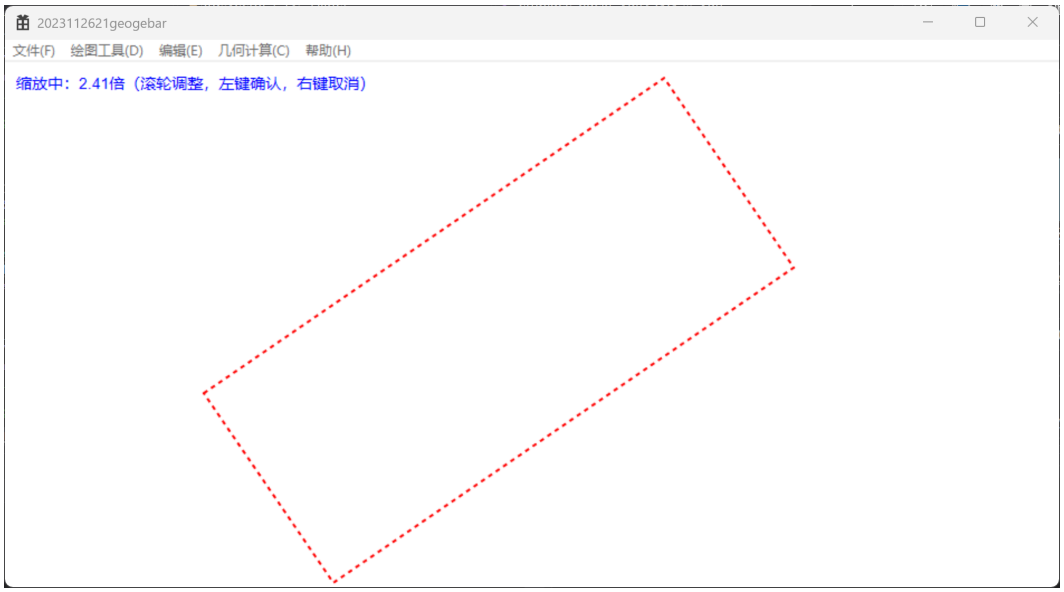


图 19: 缩放图形示例（放大）

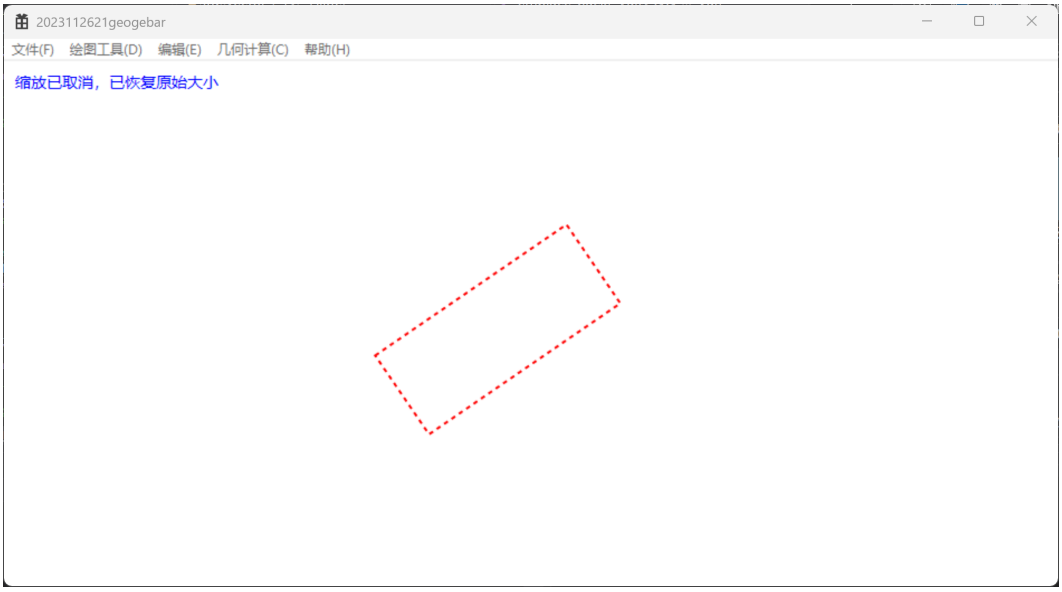


图 20: 缩放图形示例（复原大小）

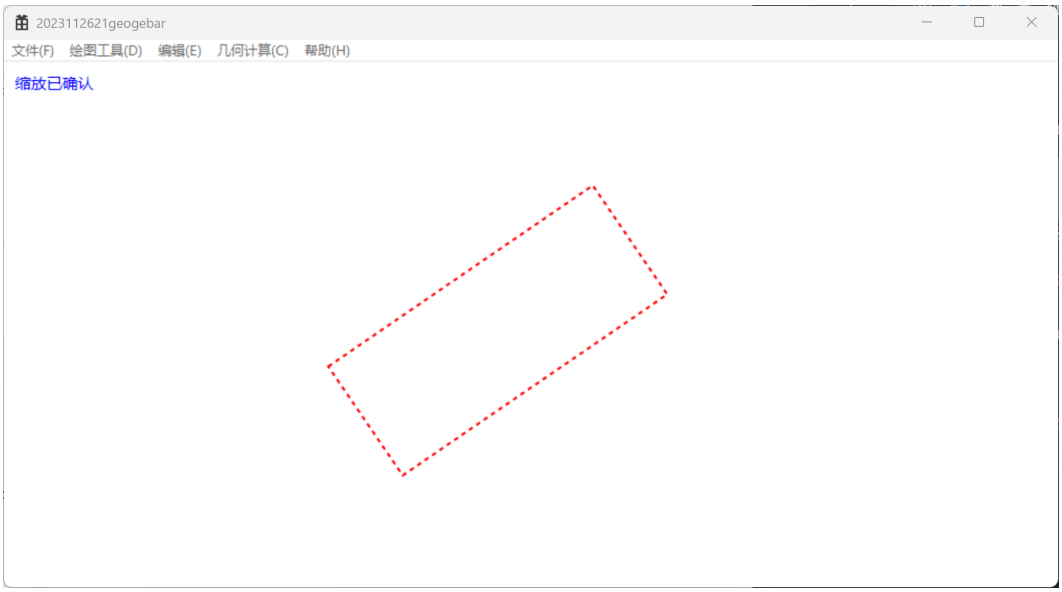


图 21: 右键取消缩放示例（确认缩放）

5. 删除图形

1. 先选中要删除的图形
2. 点击菜单” 编辑 → 删除” 或按 Delete 键
3. 图形被删除（可通过撤销功能恢复）

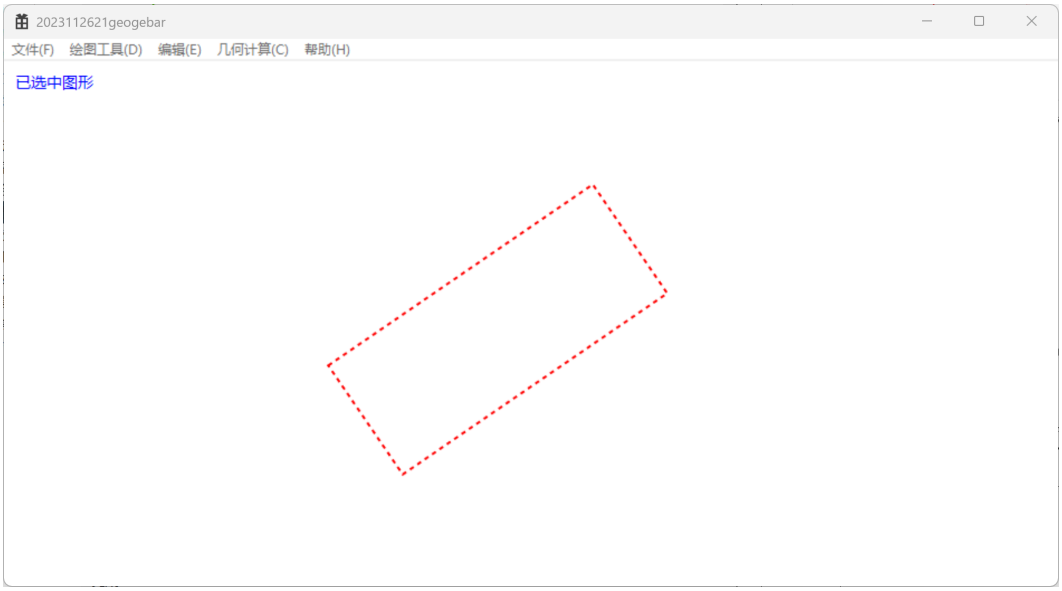


图 22: 删除图形示例（删除前）

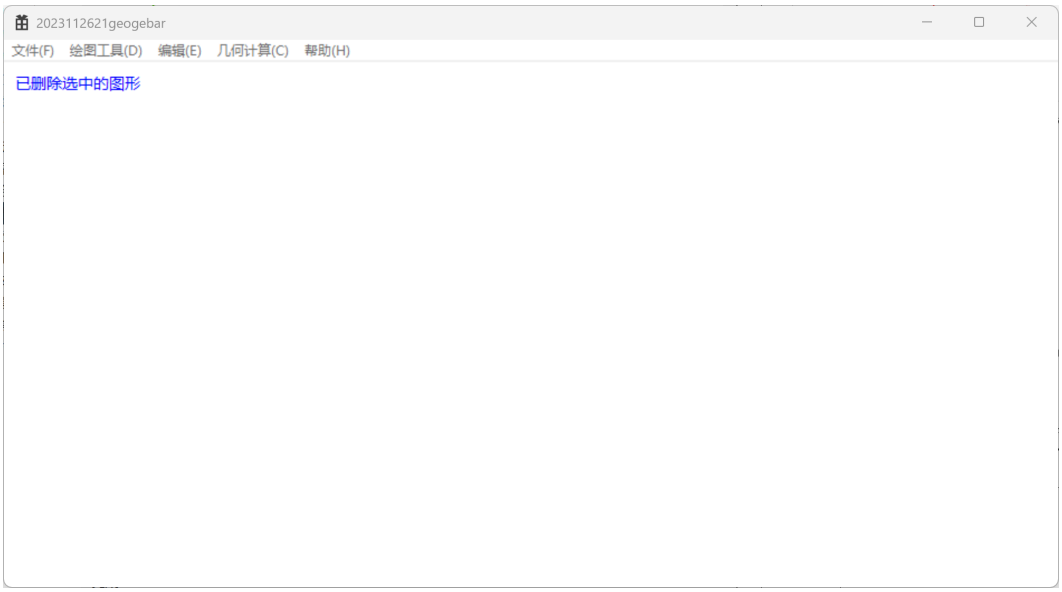


图 23: 删除图形示例（删除后）

6.4 撤销与重做操作

撤销操作

1. 点击菜单” 编辑 → 撤销” 或按快捷键 Ctrl+Z
2. 系统恢复到上一个操作状态

3. 可连续撤销最多 50 步操作

4. 当没有可撤销的操作时，菜单项显示为灰色

重做操作

1. 点击菜单” 编辑 → 重做” 或按快捷键 Ctrl+Y

2. 系统恢复到下一个操作状态

3. 只能重做已撤销的操作

4. 执行新操作后，重做历史会被清空

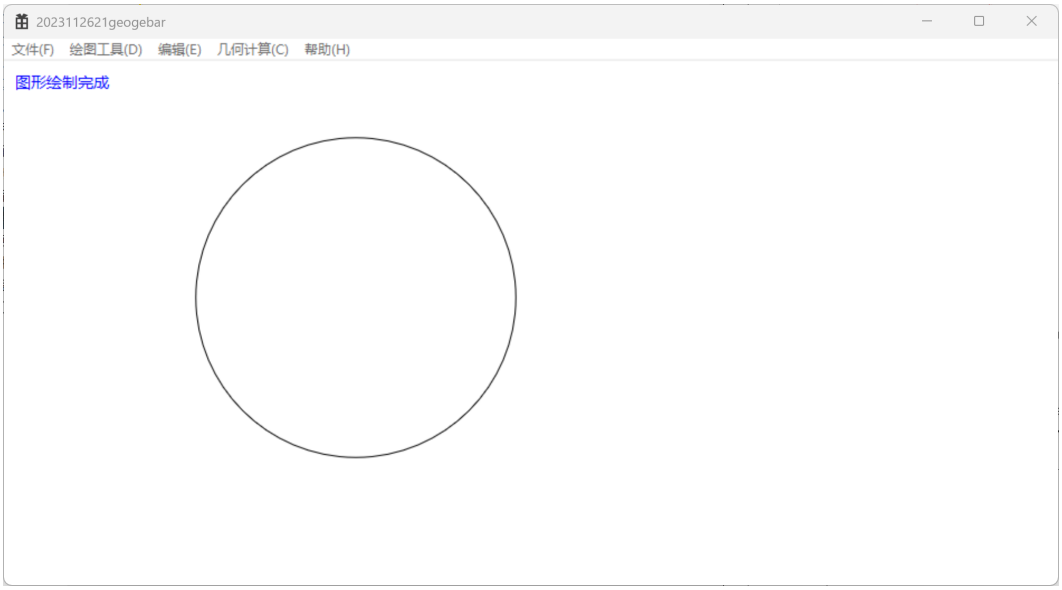


图 24: 撤销操作示例（撤销前）

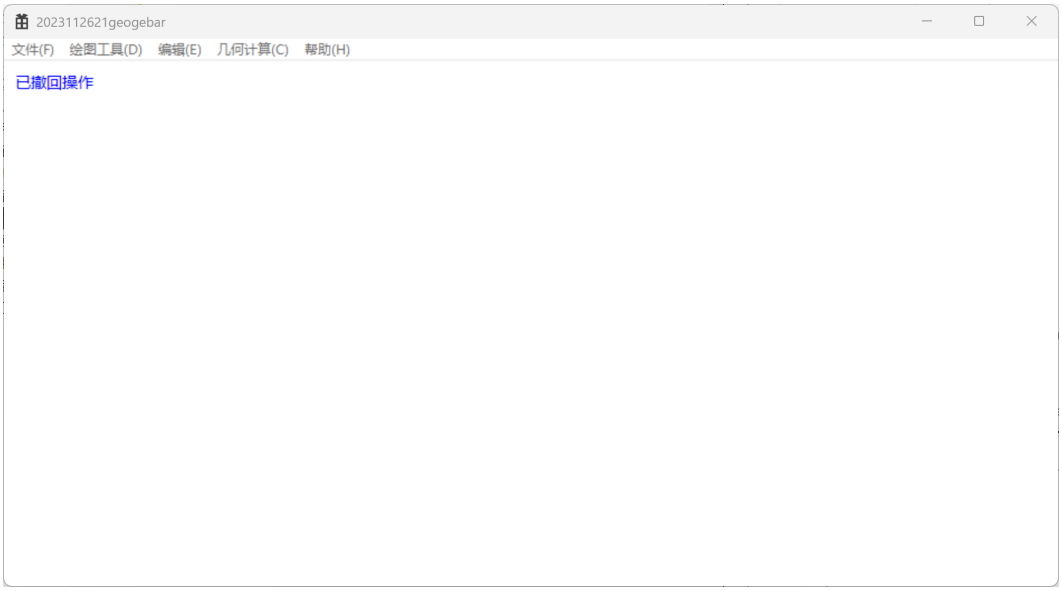


图 25: 撤销操作示例（撤销后）

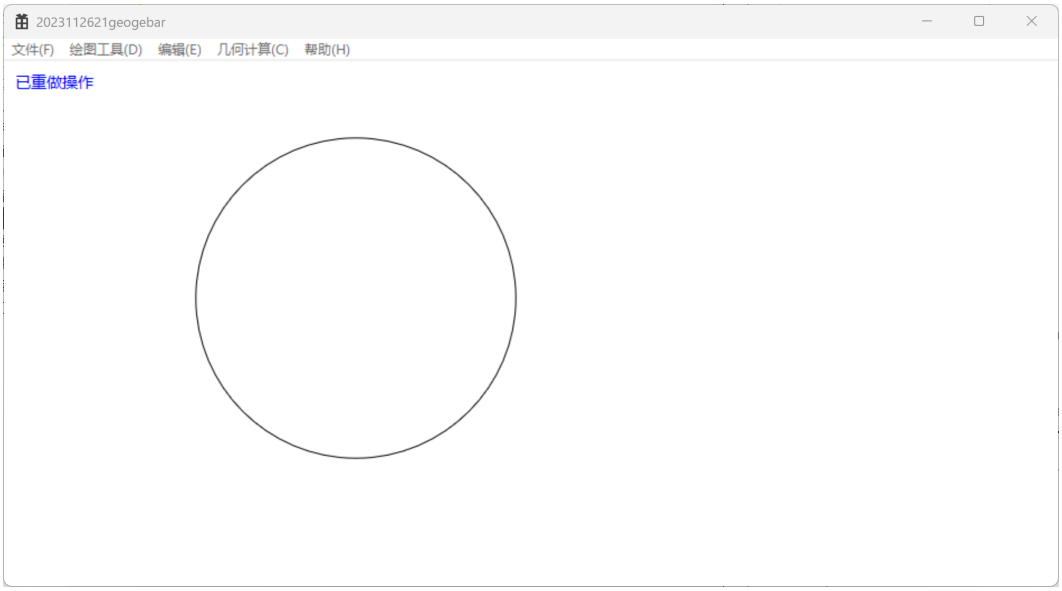


图 26: 重做操作示例

6.5 几何计算功能

1. 求交点

1. 点击菜单” 计算 → 求交”
2. 点击第一个图形（用蓝色高亮显示）

3. 点击第二个图形（用绿色高亮显示）
4. 系统自动计算并显示交点
5. 交点用红色圆点标记
6. 在消息框中显示交点坐标

支持的图形组合：

- 直线与直线：最多 1 个交点
- 直线与圆：最多 2 个交点
- 圆与圆：最多 2 个交点

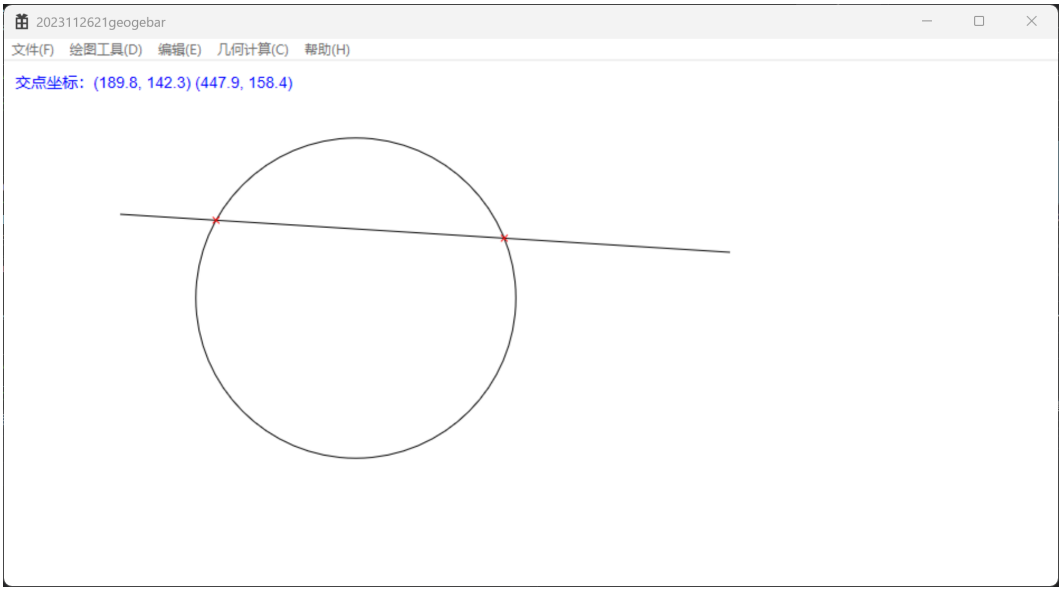


图 27: 求交点示例

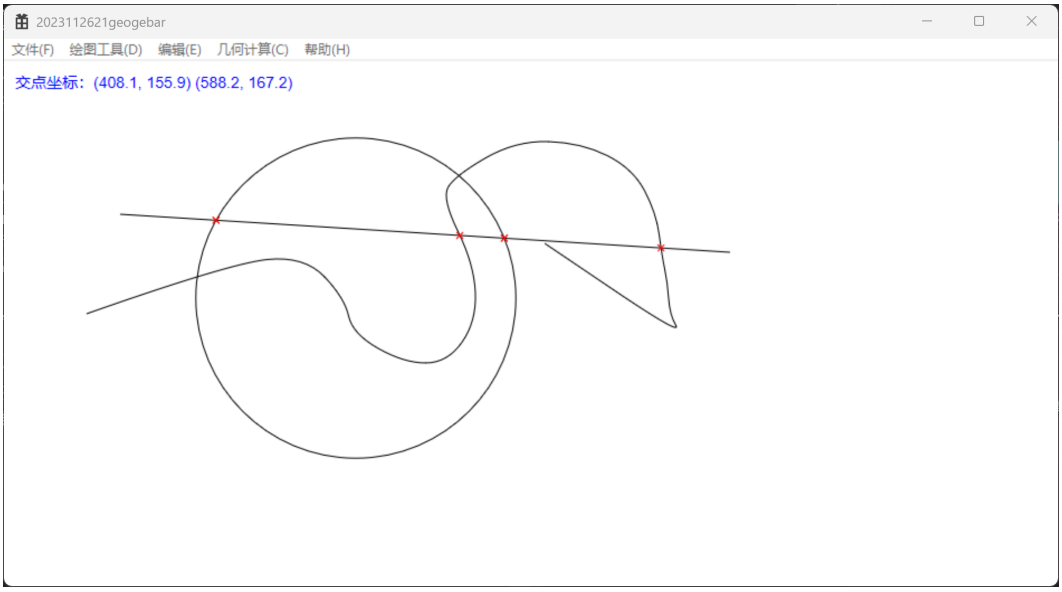


图 28: 求交点示例

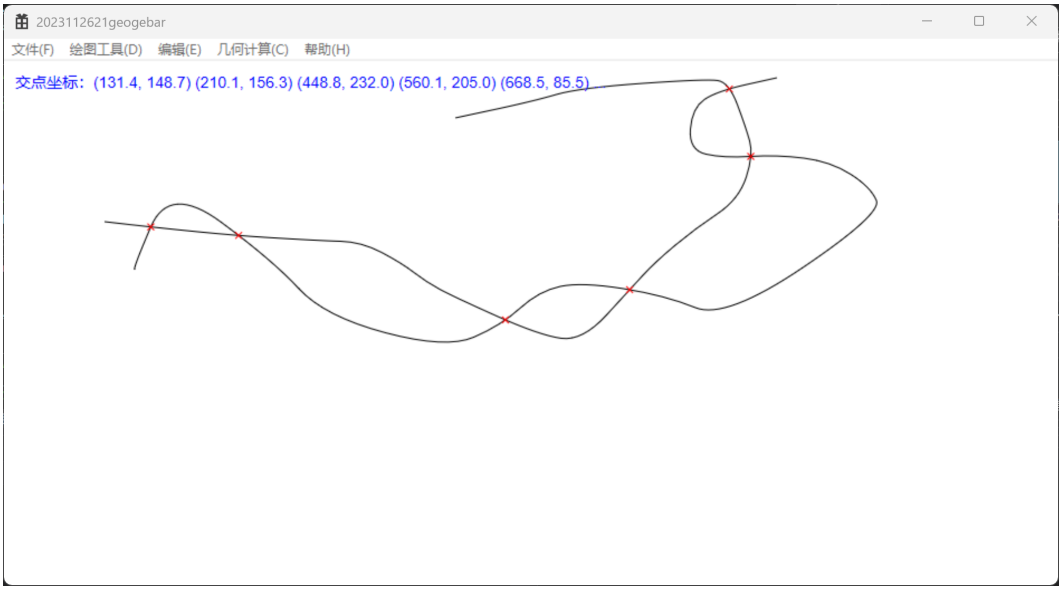


图 29: 求交点示例

2. 绘制垂线

1. 点击菜单” 计算 → 垂线”
2. 点击选择一条直线（用蓝色高亮显示）
3. 在画布上点击一个点
4. 系统自动绘制过该点的垂线（绿色虚线）

5. 垂足位置用绿色圆点标记

6. 显示垂足坐标信息

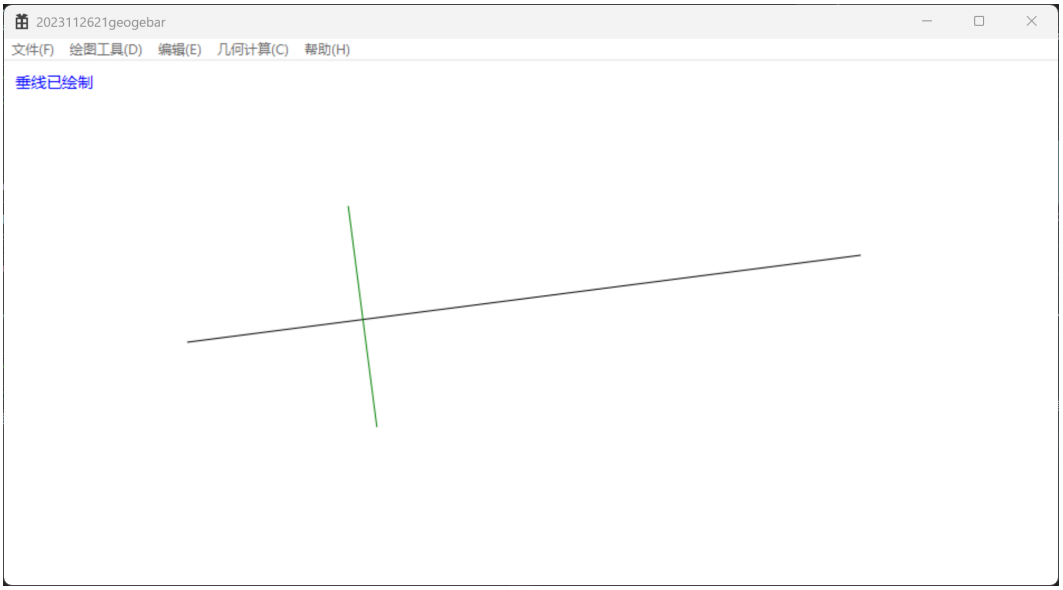


图 30: 求垂线示例

3. 显示圆心

1. 点击菜单” 计算 → 圆心”

2. 点击选择一个圆（用蓝色高亮显示）

3. 系统在圆心位置绘制红色十字标记

4. 显示圆心坐标、半径、周长和面积信息

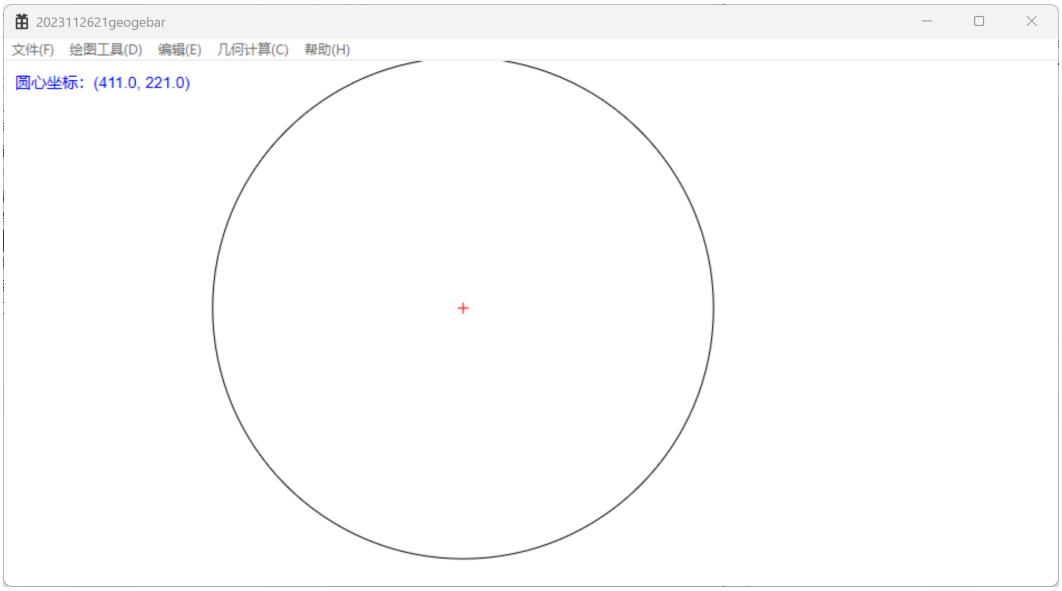


图 31: 求圆心示例

4. 绘制切线

1. 点击菜单” 计算 → 切线”
2. 在画布上点击一个外部点
3. 点击选择一个圆（用蓝色高亮显示）
4. 系统自动绘制两条切线（紫色虚线）
5. 两个切点用紫色圆点标记
6. 显示切点坐标和切线长度信息

注意事项：

- 点必须在圆外才能绘制切线
- 点在圆上时只有一条切线
- 点在圆内时无法绘制切线

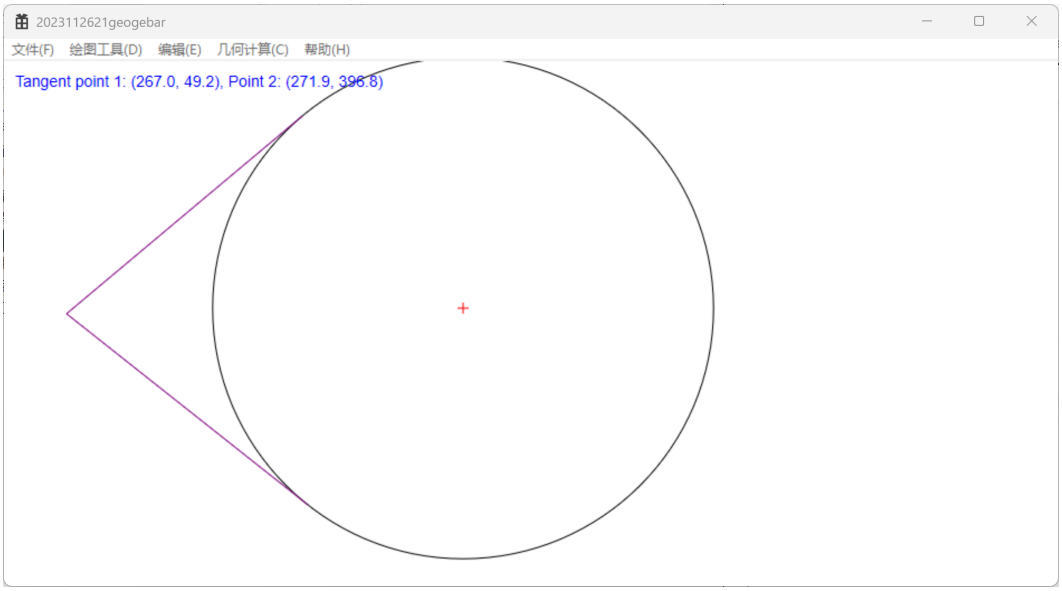


图 32: 求切线示例

6.6 文件操作

1. 新建画布

1. 点击菜单” 文件 → 新建”
2. 如果当前有未保存的内容，系统会提示是否保存
3. 确认后清空画布，开始新的绘图

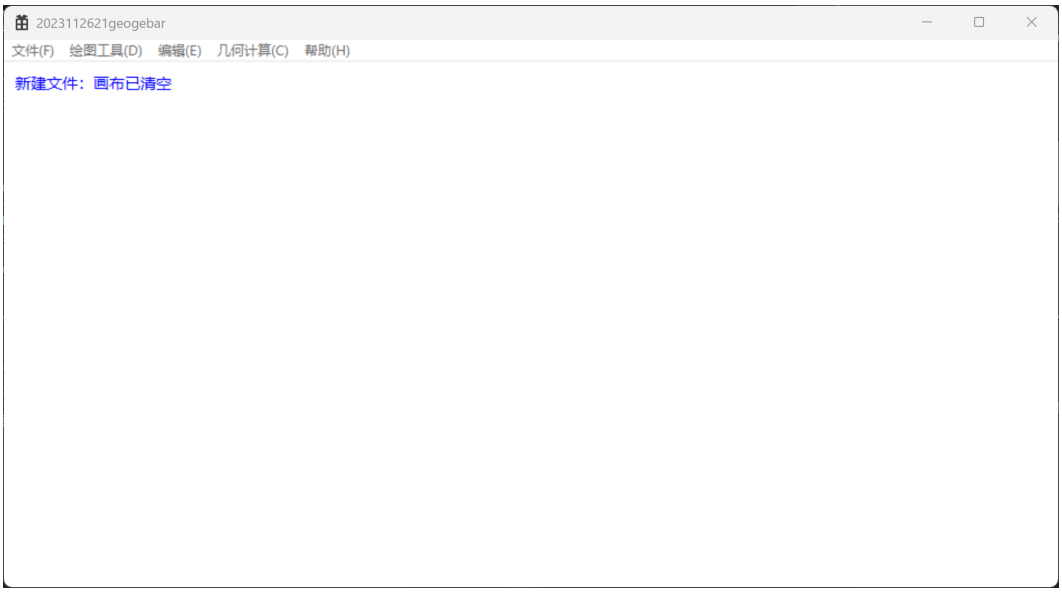


图 33: 新建画布示例

2. 保存文件

1. 点击菜单” 文件 → 保存”
2. 在弹出的保存对话框中选择保存位置
3. 输入文件名（默认扩展名为.drw）
4. 点击” 保存” 按钮
5. 系统显示” 文件保存成功” 提示

文件格式说明：

- 文件扩展名：.drw
- 文件格式：纯文本格式
- 第一行：图形总数
- 后续每行：一个图形的序列化数据

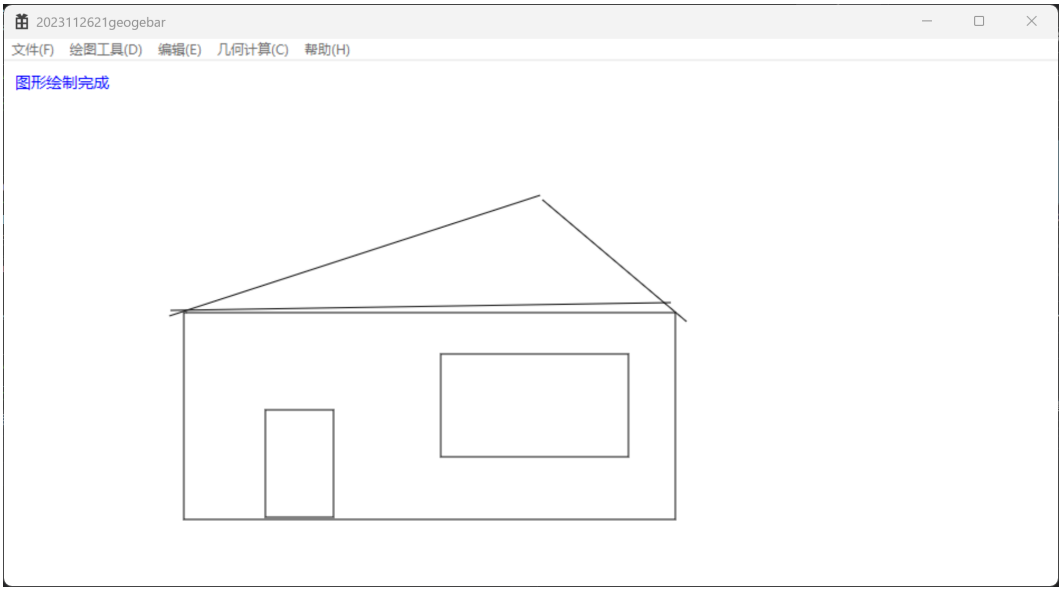


图 34: 保存示例

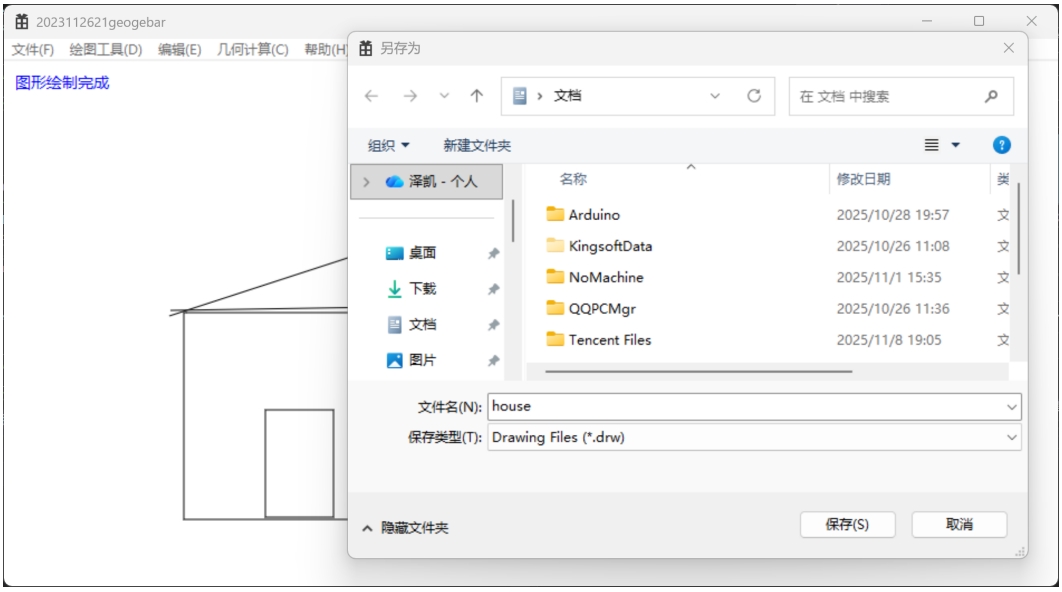


图 35: 保存示例

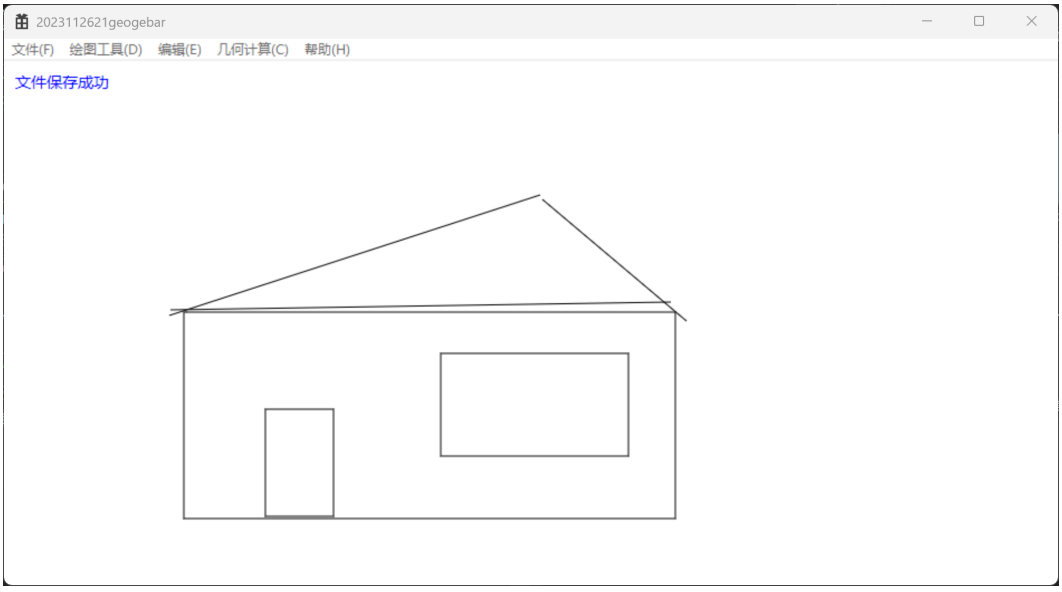


图 36: 保存示例

3. 打开文件

1. 点击菜单” 文件 → 打开”
2. 在弹出的打开对话框中选择.drw 文件
3. 点击” 打开” 按钮
4. 系统清空当前画布并加载文件中的图形

5. 显示” 文件加载成功” 提示

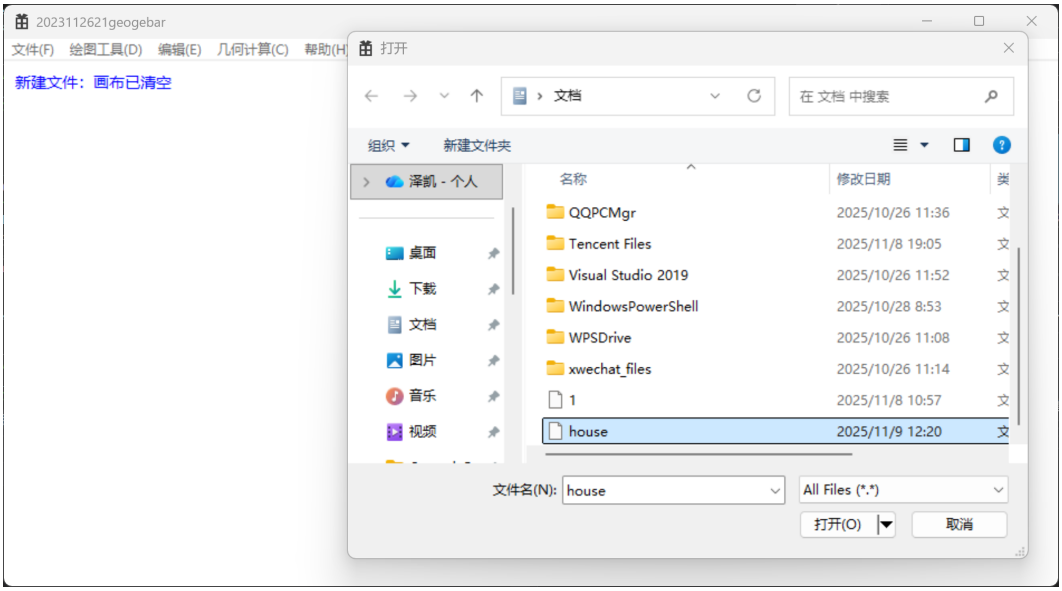


图 37: 加载示例

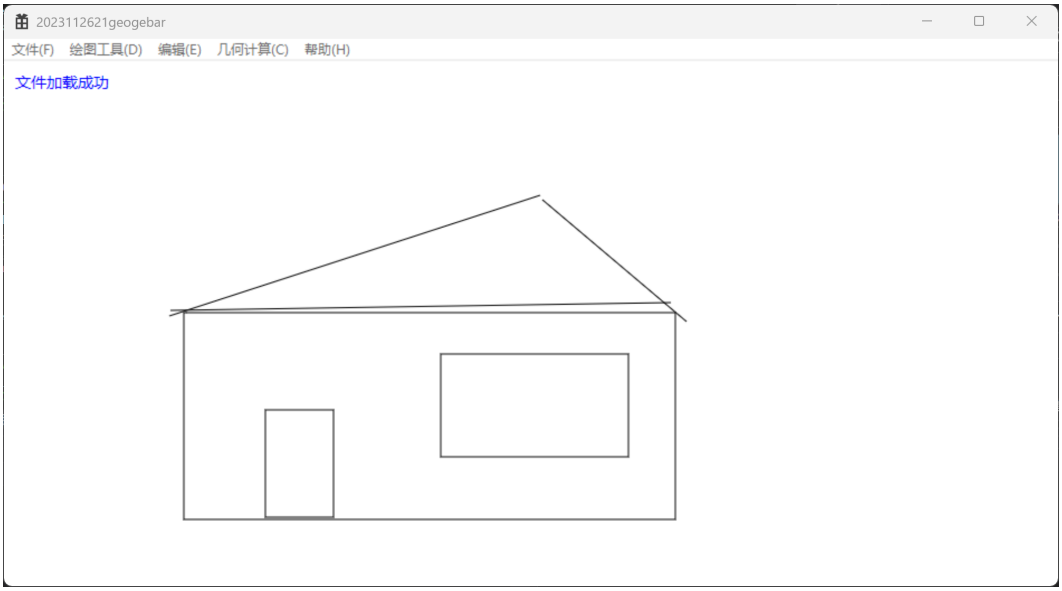


图 38: 加载示例

4. 退出程序

1. 点击菜单” 文件 → 退出” 或点击窗口关闭按钮
2. 如果有未保存的内容，系统会提示是否保存
3. 确认后程序退出

7 实验总结

通过本次实验，我成功设计并实现了一个功能完整的简单绘图软件，主要收获如下：

1. 系统设计能力：学会了如何设计一个完整的图形交互系统，包括数据结构设计、模块划分、接口定义等。掌握了面向对象的设计方法，通过继承和多态实现了灵活可扩展的图形框架。
2. 图形编程技术：深入掌握了 Direct2D 的高级应用，包括渲染目标管理、画刷样式设置、虚线绘制等技术。学会了如何实现图形的 HitTest、变换操作等核心功能。
3. 事件处理机制：理解了 Windows 消息机制，掌握了鼠标事件（点击、移动、滚轮）的处理方法。学会了如何设计状态机来管理不同的操作模式（绘制、选择、编辑等）。
4. 状态管理技术：实现了完整的撤销/重做功能，掌握了使用双栈结构管理历史状态的方法。通过序列化/反序列化实现了图形的深拷贝，理解了状态独立性的重要性。
5. 交互设计优化：改进了旋转和缩放的交互方式，使用鼠标滚轮配合实时预览和取消功能，大大提升了用户体验。学会了如何设计直观、友好的用户交互界面。
6. 几何算法应用：实现了多种几何计算算法，包括点到线距离、直线求交、圆-圆求交、切线计算等。将数学知识应用到实际编程中，提高了算法实现能力。
7. 文件格式设计：学会了如何设计简单的文件格式，实现了图形数据的序列化和反序列化。理解了数据持久化的基本原理。
8. 调试与测试：通过大量的测试和调试，提高了发现和解决问题的能力。学会了如何系统地测试软件的各项功能。

遇到的主要问题及解决方案：

- 问题 **1**：图形选择的 HitTest 精度问题。解决：为不同类型的图形设计了不同的 HitTest 算法，并设置了合理的容差值。
- 问题 **2**：图形旋转时的中心点计算。解决：为每种图形实现了 GetCenter() 方法，根据图形类型计算几何中心。
- 问题 **3**：虚线绘制的实现。解决：使用 Direct2D 的 StrokeStyle 功能，创建虚线样式并应用到选中图形的绘制。

- 问题 **4**：文件格式的设计。解决：采用简单的文本格式，每种图形一行，便于调试和扩展。
- 问题 **5**：旋转和缩放的交互体验。解决：改用鼠标滚轮进行调整，配合实时预览和取消功能，提供更好的用户体验。用户可以在确认前看到效果，不满意可以右键取消。
- 问题 **6**：撤销/重做的深拷贝问题。解决：通过序列化和反序列化实现图形的深拷贝，确保状态独立性。每次保存状态时都创建完全独立的图形副本。

本实验不仅巩固了图形编程的基础知识，更重要的是培养了完整系统的设计与实现能力。通过实践，深刻理解了软件工程的基本原则，为后续更复杂的图形应用开发奠定了坚实的基础。